

01 AI foundations



02 Generative AI



03 AI development options



04 AI development workflow



In the previous module, you learned about different options to develop an AI project on Google Cloud, from a ready-to-use approach like pre-trained APIs, to low or no-code solutions like AutoML, and to DIY solutions, such as custom training.

Then, **how** do you build an ML model step-by-step?

You'll find out in this module.



Agenda



- | | |
|----|---------------------------------|
| 01 | ML workflow |
| 02 | Data preparation |
| 03 | Model development |
| 04 | Model serving |
| 05 | MLOps and workflow automation |
| 06 | Lab: AutoML |
| 07 | How a machine learns (optional) |
| 08 | Summary |

You begin with an overview of the ML workflow, then dive into each of the three workflow stages: from data preparation to model development and finally model serving.

Next, you investigate machine learning operations, or MLOps, which takes ML models from development to production in the back end. You'll be shown an example of how to build a pipeline to automate the production using Vertex AI Pipelines.

A hands-on lab will then help you walk through the three stages to build an ML model with *AutoML on Vertex AI*.

Gaining a solid grasp of ML terminology requires a clear understanding of how a neural network learns. This module offers an optional lesson that delves into the neural network's learning process, along with the key terminologies. If you're already familiar with the ML theories, feel free to skip this lesson.

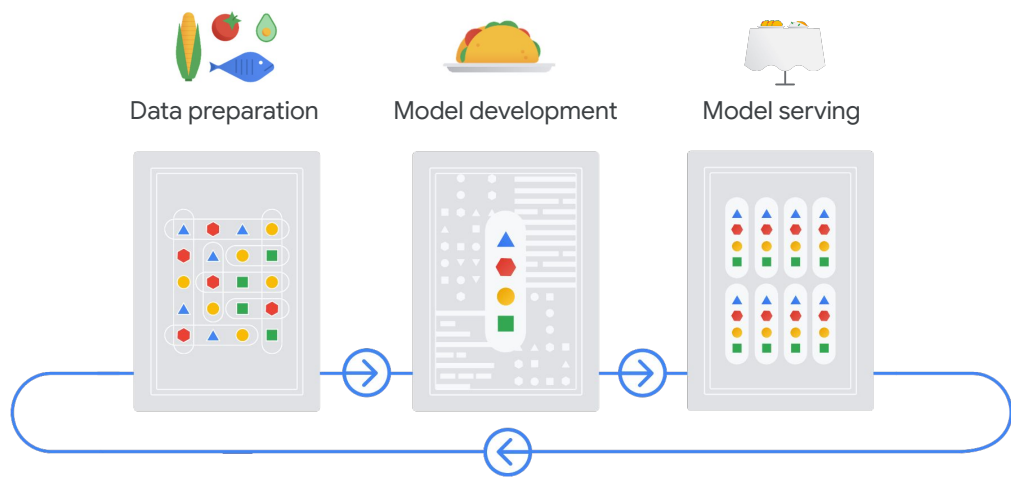


Agenda



- | | |
|----|---------------------------------|
| 01 | ML workflow |
| 02 | Data preparation |
| 03 | Model development |
| 04 | Model serving |
| 05 | MLOps and workflow automation |
| 06 | Lab: AutoML |
| 07 | How a machine learns (optional) |
| 08 | Summary |

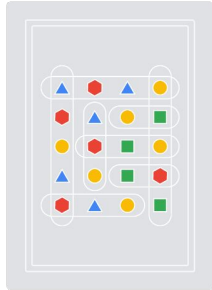
Let's get started with the ML workflow.



Building an ML model is actually not too different from serving food in a restaurant. You start by preparing raw ingredients and finish by serving the dishes on the table.

There are three main stages to the ML workflow with Vertex AI.

Data preparation



Two steps

- ✓ Upload data.
- ✓ Engineer features.

Data types

- ✓ Streaming versus batch data.
- ✓ Structured versus unstructured data.

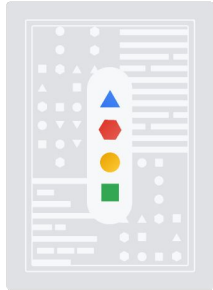
The first stage is data preparation, which includes two steps: data uploading and feature engineering.

A model needs a large amount of data to learn from. The quality and quantity of the data decide how much and how well the machine learns.

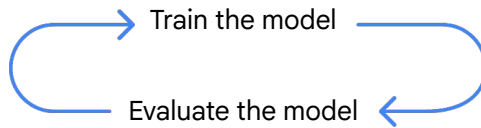
The data used in machine learning can be real-time streaming data or batch data. The data can also be structured or unstructured.

Structured data is data that can be easily stored in tables, such as numbers and text. Unstructured data is data that cannot be easily stored in tables, such as images and videos.

Model development



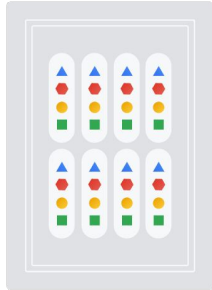
A model needs a tremendous amount of iterative training.



The second stage of the ML workflow is model development.

A model needs a tremendous amount of iterative training. This is when training and evaluation form a cycle to train a model, then evaluate the model, and then train the model some more.

Model serving

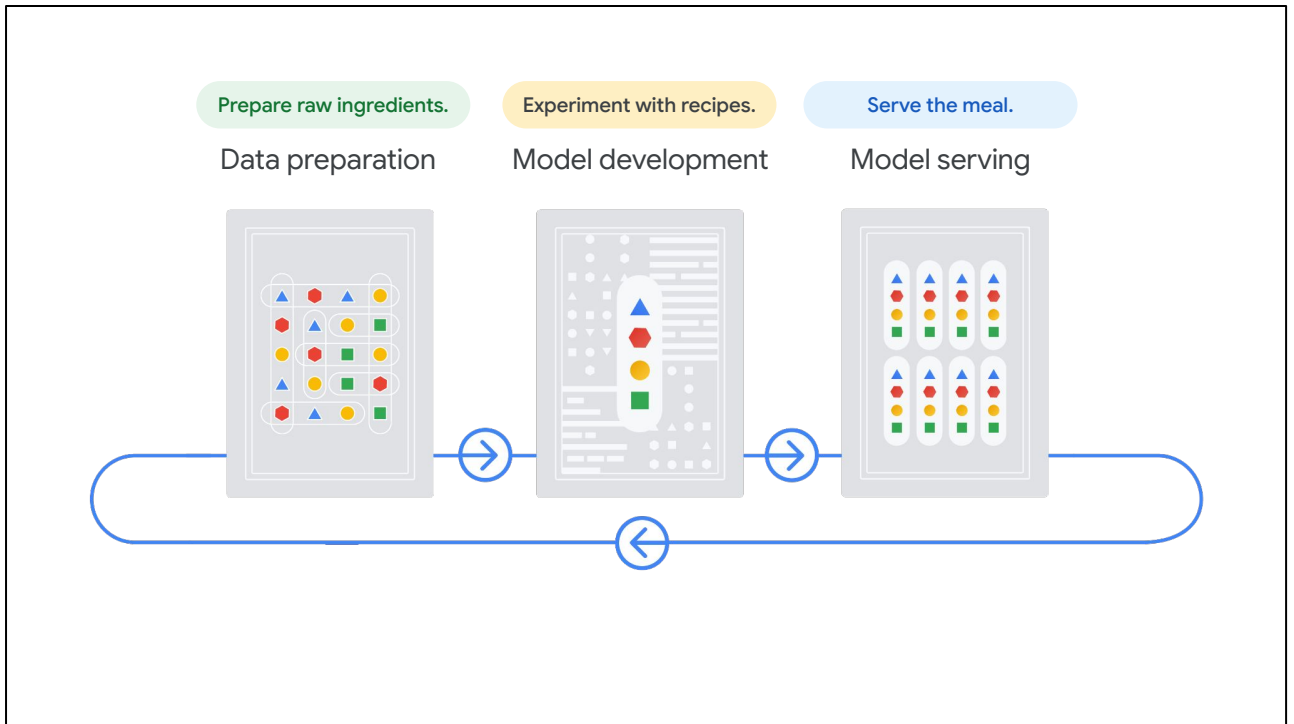


A model needs to actually be used in order to predict results.

- ✓ Deploy the model.
- ✓ Monitor the model.

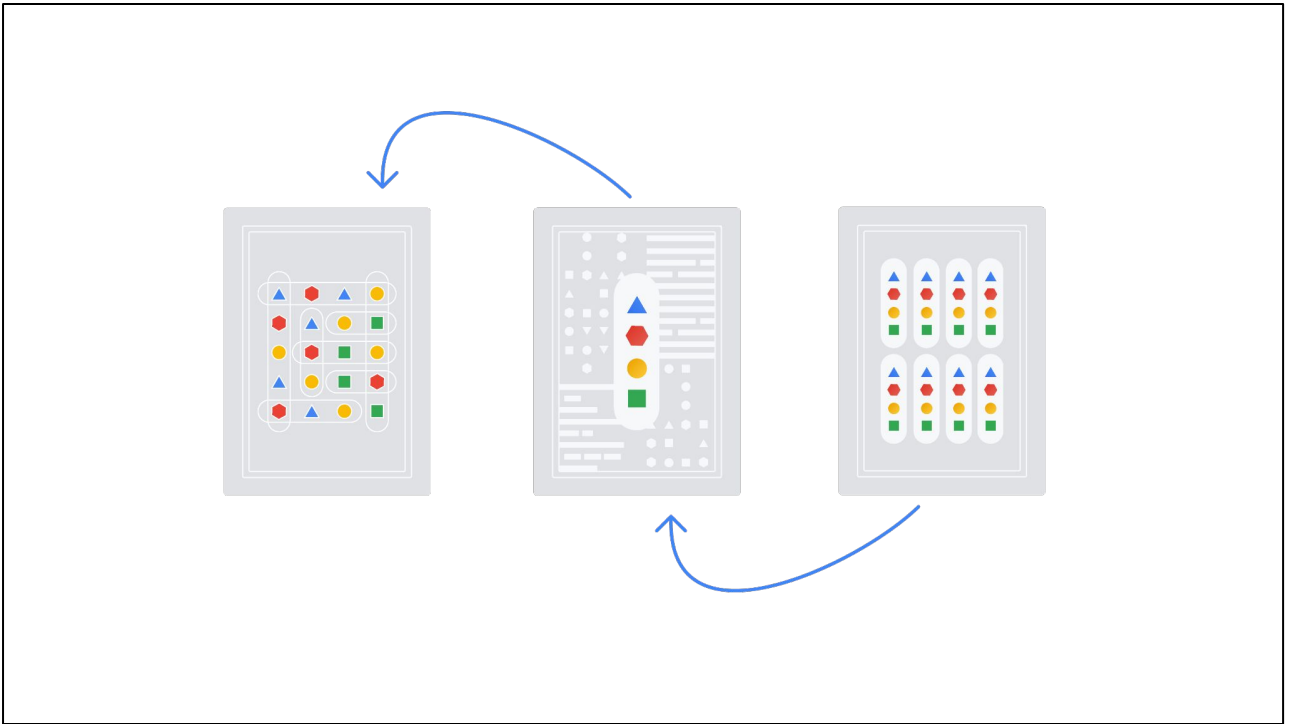
The third and final stage is model serving.

A model needs to actually be used in order to predict results. This is when the machine learning model is deployed and monitored. If you don't move an ML model into production, it has no use and remains only a theoretical model.



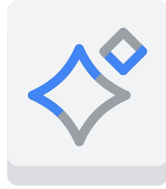
Compare this process to serving food in a restaurant.

Data preparation is when you prepare the raw ingredients, model development is when you experiment with different recipes, and model serving is when you finalize the menu to serve the meal to customers.



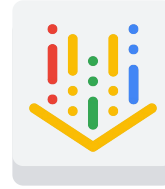
Now, it's important to note that an ML workflow isn't linear, it's iterative. For example, during model training, you might need to return to the raw data and generate more useful features to feed the model.

When monitoring the model during model serving, you might find data drifting or the accuracy of your prediction might suddenly drop. You might need to check the data sources and adjust the model parameters. Fortunately, these steps can be automated with MLOps. You'll learn more about this later in this module.



AutoML with Vertex AI

- ✓ No-code approach
- ✓ Configuration through UI
- ✓ User friendly





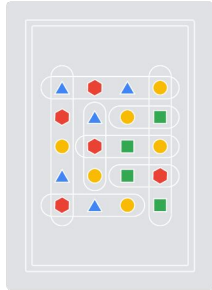
Agenda



- 01 ML workflow
- 02 Data preparation
- 03 Model development
- 04 Model serving
- 05 MLOps and workflow automation
- 06 Lab: AutoML
- 07 How a machine learns (optional)
- 08 Summary

Let's start with the first stage of ML workflow, **data preparation**.

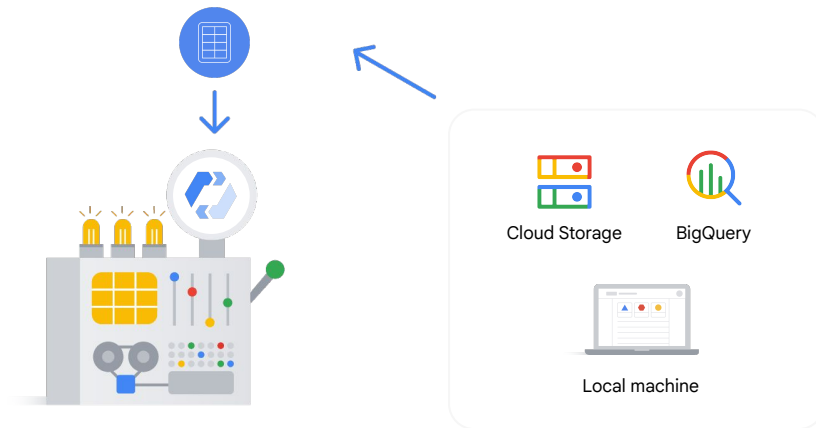
Data preparation



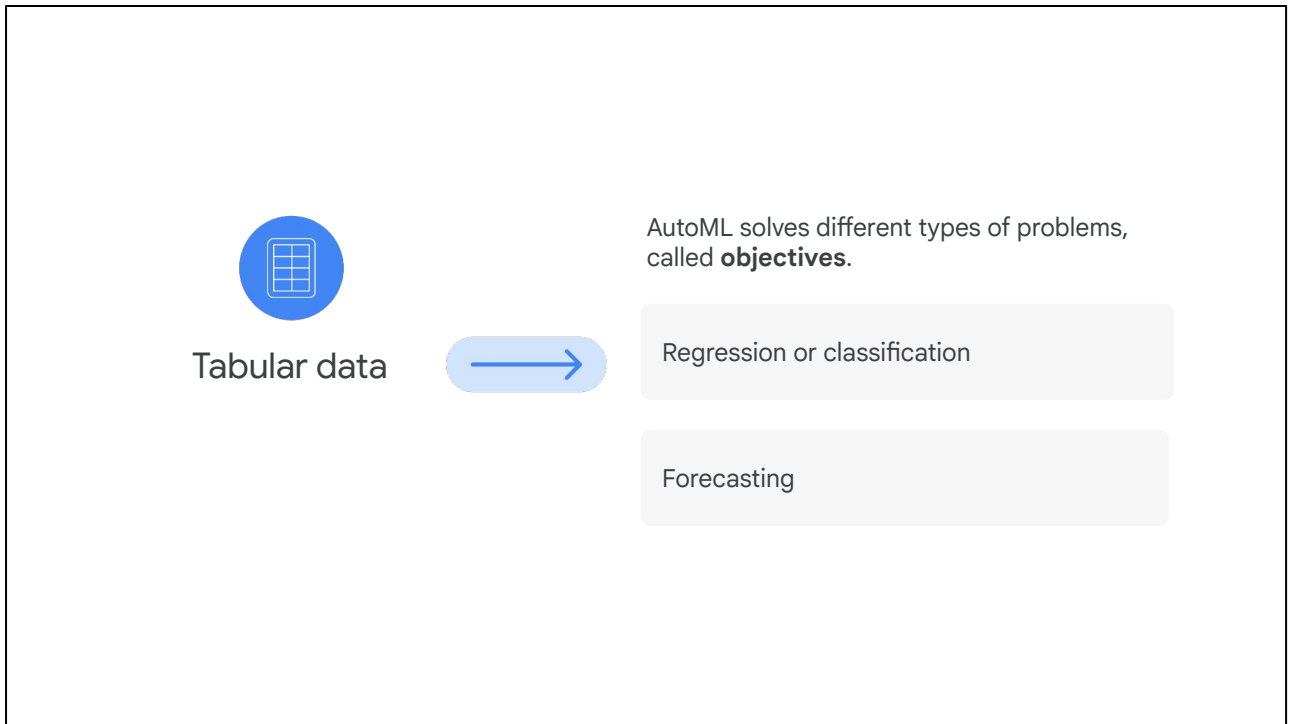
01 Upload data.

02 Engineer features.

During this stage, you must *upload data* and then prepare it for model training with *feature engineering*.



The data can come from Cloud Storage, BigQuery, or even your local machine.

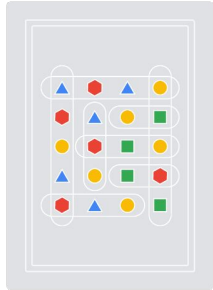


AutoML now mainly supports *tabular* data by solving different types of problems, called *objectives*.

For *tabular data*, you can train the model to solve regression, classification, or forecasting problems.

Forecasting is vital to many industries like retail. To learn more about how to build a forecasting model, please check the course titled [Introduction to Vertex Forecasting and Time Series in Practice](#) in the reading list.

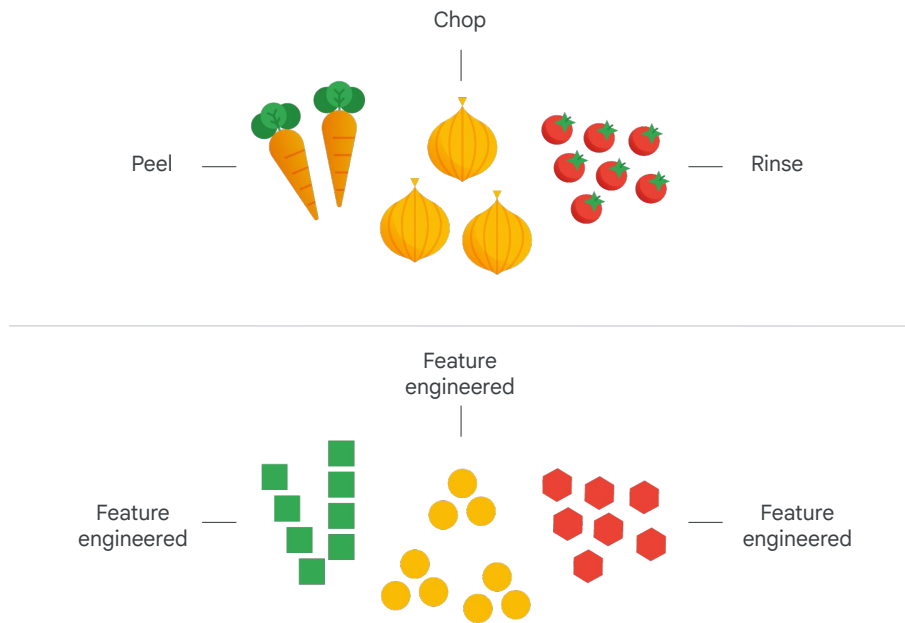
Data preparation



01 Upload data.

02 Engineer feature.

After the data is uploaded to AutoML, the next step is preparing it for model training with feature engineering.



Imagine you're preparing a meal. Your data is like your ingredients, such as carrots, onions, and tomatoes. Before you start cooking, you'll need to peel the carrots, chop the onions, and rinse the tomatoes.

This is what feature engineering is like: the data must be processed before the model starts training.

A **feature** refers to a factor that contributes to the prediction.

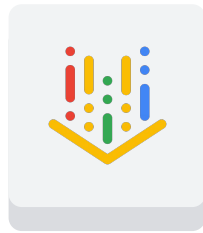
Independent variable

Column in a table

A feature refers to a factor that contributes to the prediction.

It's like an independent variable in statistics or a column in a table.

Preparing features can be both challenging and tedious.



Vertex AI
Feature Store

A centralized repository to manage, serve, and share features

✓ Real-time (online) serving

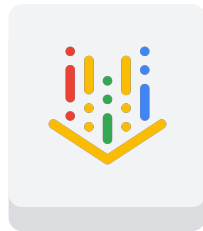
✓ Batch (offline) serving

● At scale, low latency

To help, Vertex AI provides a service called *Vertex AI Feature Store*, which is a centralized repository to manage, serve, and share features.

It aggregates the features from different sources in *BigQuery* and makes them available for both real-time (often called online) and batch (often called offline) serving.

Vertex AI automates the feature aggregation to scale the process with low latency.



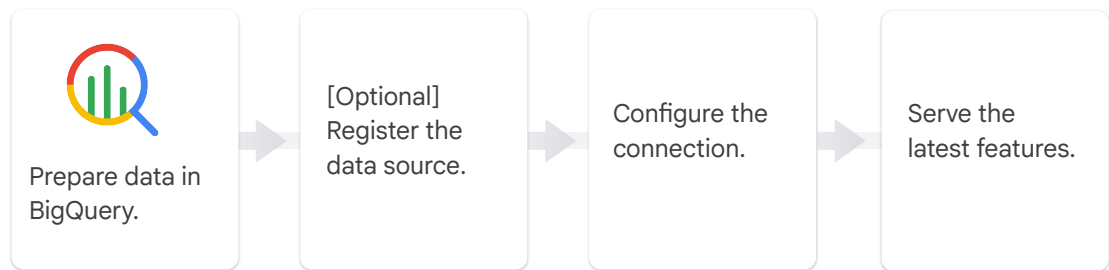
Vertex AI
Feature Store

Ready for generative AI

- ✓ Manage and serve embeddings.
- ✓ Retrieve similar items.
- In real time

Additionally, Vertex AI Feature Store is ready for generative AI.

- It can manage and serve embeddings, which are the data sources in gen AI.
- It also supports retrieving similar items in real time, ensuring low latency.



The workflow to set up and start real-time (online) serving using Vertex AI Feature Store can be summarized as follows:

1. Prepare the data source in BigQuery.
2. Optional: Register the data sources by creating feature groups and features.
3. Configure the connection by creating a feature view to define which features to copy from your data source into the online store for real-time serving.
4. Serve the latest feature values online from a feature view.



Benefits of Vertex AI Feature Store



Features are shareable for training and serving.



Features are reusable.



Features are scalable.



Features are easy to use.

So what are the benefits of Vertex AI Feature Store?

- First, **features are shareable** for training and serving. They are managed and served from a central repository, maintaining consistency across your organization.
- Second, **features are reusable**. This helps to save time and reduces duplicated efforts.
- Third, **features are scalable**. They automatically scale to provide low-latency serving, so you can focus on developing the logic to create them without worrying about deployment.
- And fourth, **features are easy to use**. Vertex AI Feature Store is built on an easy-to-navigate user interface.



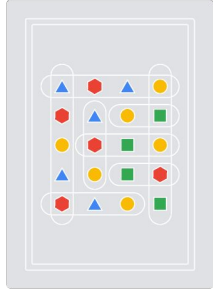
Agenda



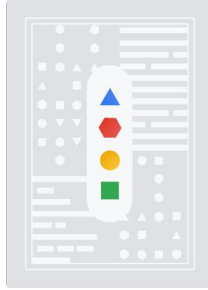
- | | |
|----|---------------------------------|
| 01 | ML workflow |
| 02 | Data preparation |
| 03 | Model development |
| 04 | Model serving |
| 05 | MLOps and workflow automation |
| 06 | Lab: AutoML |
| 07 | How a machine learns (optional) |
| 08 | Summary |

Let's advance to the second stage, model development, where you train the model and evaluate the result.

Model development



Prepare raw ingredients.



Experiment with recipes.

01 Train model.

02 Evaluate model.

Now that our data is ready—which, if you return to the cooking analogy, is the ingredients—it's time to train the model. This is like experimenting with recipes.

This stage involves two steps: model training, which is like cooking the recipe, and model evaluation, which is like testing how good the meal is. This process might be iterative.

Train new model

- 1 Training method
- 2 Model details
- 3 Training options
- 4 Compute and pricing

START TRAINING CANCEL

Dataset *
retailSales

Objective *
Classification

Model training method

☒ AutoML

☐ Custom training (advanced)

CONTINUE

To set up an ML model, you need to specify a few things. Please note that the user interface may not be exactly the same as the product is evolving rapidly. However, similar features will remain.

First of all is the **training method**, where you tell Vertex AI:

- The dataset you just uploaded from the preparation stage.
- Depending on the data type, whether it is tabular, image, text, or video, you specify the training objective. This is the goal of the model training and the task you want to solve.
- Then you choose the training method: AutoML, without code, or custom training, using code.

Train new model

✓ Training method

2 Model details

3 Training options

4 Compute and pricing

START TRAINING

CANCEL

☒ Train new model

☐ Train new version

Name *

retailSales

Description

Target column *

☐

✓ ADVANCED OPTIONS

CONTINUE

The next step is to determine the training details. For example, if you are training the model to solve a supervised learning problem such as regression and classification, you must choose the target column from your dataset.

Train new model

✓ Training method

✓ Model details

3 Training options

4 Compute and pricing

START TRAINING

CANCEL

i

×

Filter

●

Column name

Transformation

BigQuery

✓

☐

☐

☐

☐

✓ ADVANCED OPT

CONTINUE

Customize transformation options

☒ Automatic

☐ Categorical

☐ Timestamp

APPLY

In training options, you can choose certain features to participate in the training and transform the data type if needed.

Train new model

- ✓ Training method
- ✓ Model details
- ✓ Training options
- 4 Compute and pricing

START TRAINING CANCEL

Budget *

1

Maximum node hours

Estimated completion: May 4, 2023 1 PM GMT+2

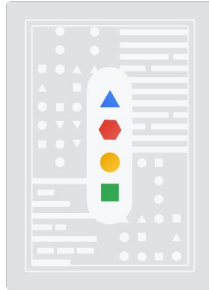
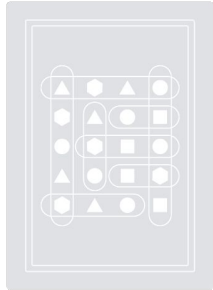
✓

Enable early stopping

Finally, you specify the budget and pricing, and then click **Start training**. AutoML will train the model for you and choose the best performed models among thousands of others.

Do you recall the powerful technologies behind AutoML? Right, the credit there goes to *neural architecture search* and *transfer learning*.

Model development

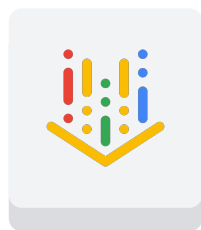


01 Train model.

02 Evaluate model.

Experiment with recipes

While you are experimenting with a recipe, you need to keep tasting it to ensure that it meets expectations. This is the *evaluation* portion of the model development stage.



Vertex AI

Evaluation metrics

Confusion matrix

Recall

Precision

Vertex AI provides extensive evaluation metrics to help determine a model's performance. Let's focus on the metrics of recall and precision when evaluating the performance of classification models. To do this, you'll use a confusion matrix.

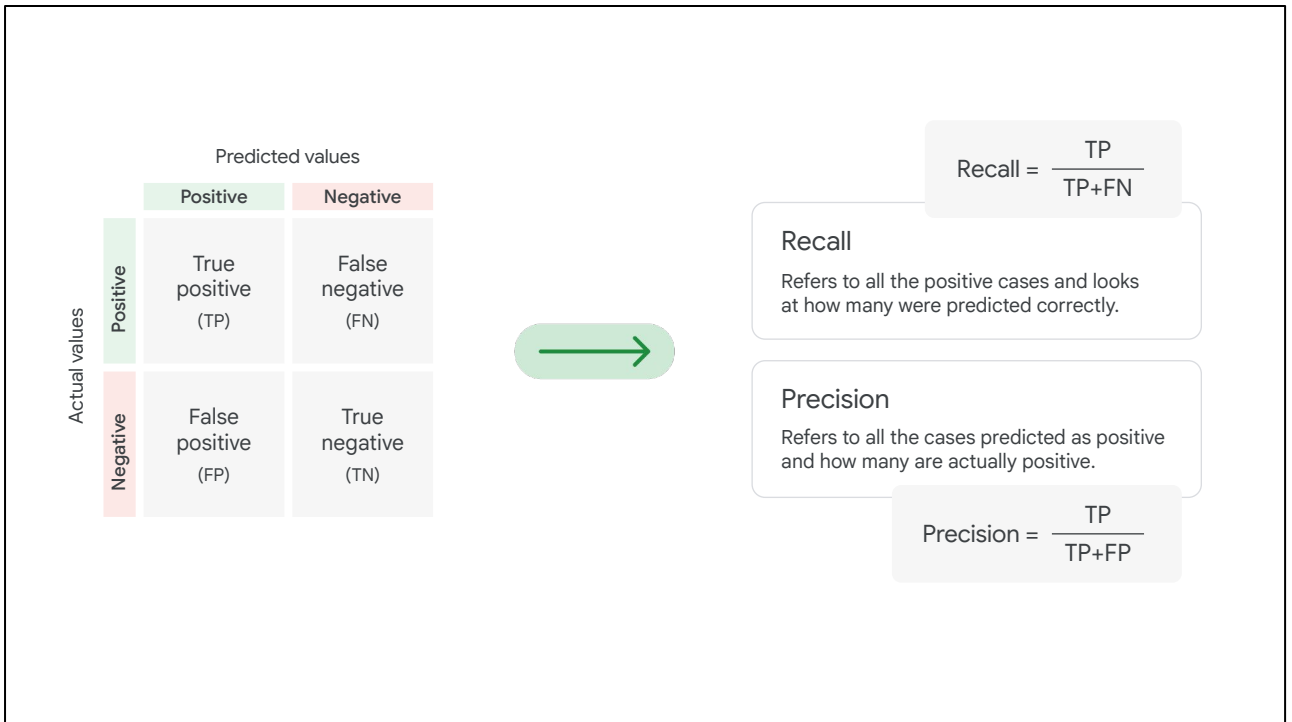
		Predicted values	
		Positive (cat)	Negative (not cat)
Actual values	Positive (cat)	True positive  This is a cat.	False negative  This is not a cat. Type 2 error
	Negative (not cat)	False positive  This is a cat. Type 1 error	True negative  This is not a cat.

To do this, you'll use a confusion matrix. A **confusion matrix** is a specific performance measurement for machine learning classification problems.

It's a table with combinations of **predicted** and **actual** values. To keep things simple, we assume the output includes only two classes.

Let's explore an example:

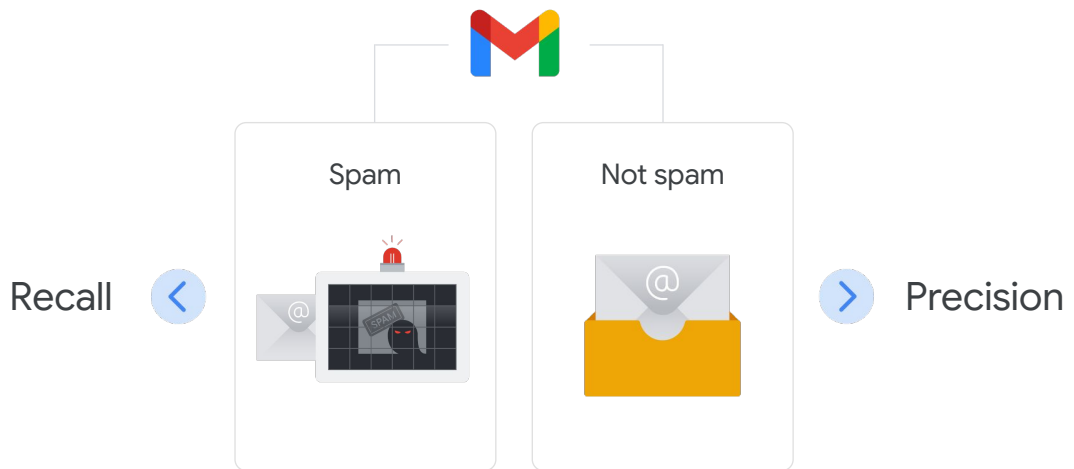
- The first is **true positive**, which can be interpreted as, "The model predicted positive, and that's true." The model predicted that this **is** an image of a cat, and it actually is.
- The opposite of that is **true negative**, which can be interpreted as, "The model predicted negative, and that's true." The model predicted that the image is **not** a cat, and it actually isn't.
- Then there is **false positive**, otherwise known as a Type 1 Error, which can be interpreted as, "The model predicted positive, and that's false." The model predicted that the image is a cat but it actually **isn't**.
- Finally, there is **false negative**, otherwise known as a Type 2 Error, which can be interpreted as, "The model predicted negative, and that's false." The model predicted that the image is **not** a cat, but it actually is.



A confusion matrix is the foundation for many other metrics used to evaluate the performance of a machine learning model.

Let's look at the two popular metrics, recall and precision, that you will encounter in the lab:

- **Recall** refers to all the positive cases, and looks at how many were predicted correctly. This means that recall is equal to the **true positives**, divided by the sum of the **true positive** and **false negatives**.
- **Precision** refers to all the cases predicted as positive, and how many are actually positive. This means that precision is equal to the **true positives**, divided by the sum of the **true positives** and **false positives**.

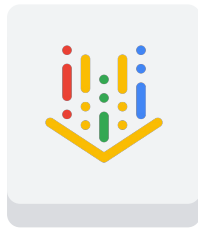


Precision and recall are often a trade-off. Depending on your use case, you might need to optimize for one or the other.

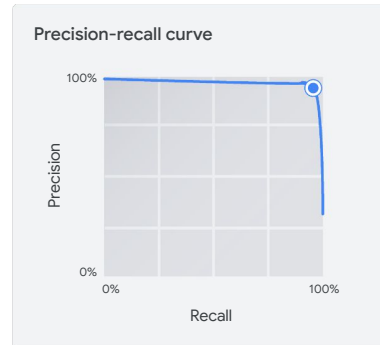
Consider a classification model where Gmail separates emails into two categories: spam and not-spam.

If the goal is to catch as many potential spam emails as possible, Gmail might want to prioritize recall.

In contrast, if the goal is to only catch the messages that are definitely spam without blocking other emails, Gmail might want to prioritize precision.

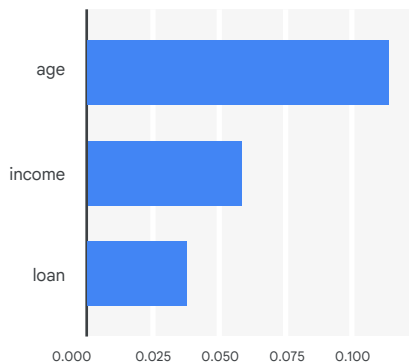


Vertex AI



Vertex AI visualizes the precision-recall curve, so it can be adjusted based on the problem that needs to be solved. You'll get the opportunity to practice adjusting precision and recall in the AutoML lab.

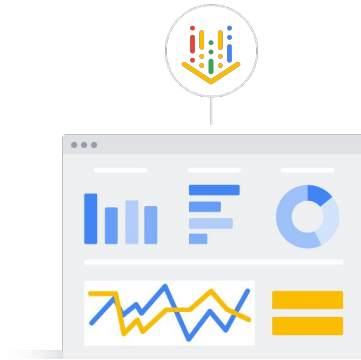
- ✓ Confusion matrix
- ✓ Feature importance



In addition to the confusion matrix and the metrics generated to measure recall and precision, the other useful measurement is **feature importance**.

In Vertex AI, feature importance is displayed through a bar chart to illustrate how each feature contributes to a prediction. The longer the bar, or the larger the numerical value associated with a feature, the more important it is. This information helps decide which features are included in a machine learning model to predict the goal.

You will observe the feature importance chart in the lab as well.



Explainable AI

Feature importance is just one example of Vertex AI's comprehensive machine learning functionality called Explainable AI. Explainable AI is a set of tools and frameworks to help understand and interpret predictions made by machine learning models. Please check the reading list if you want to know about Explainable AI on Google Cloud.

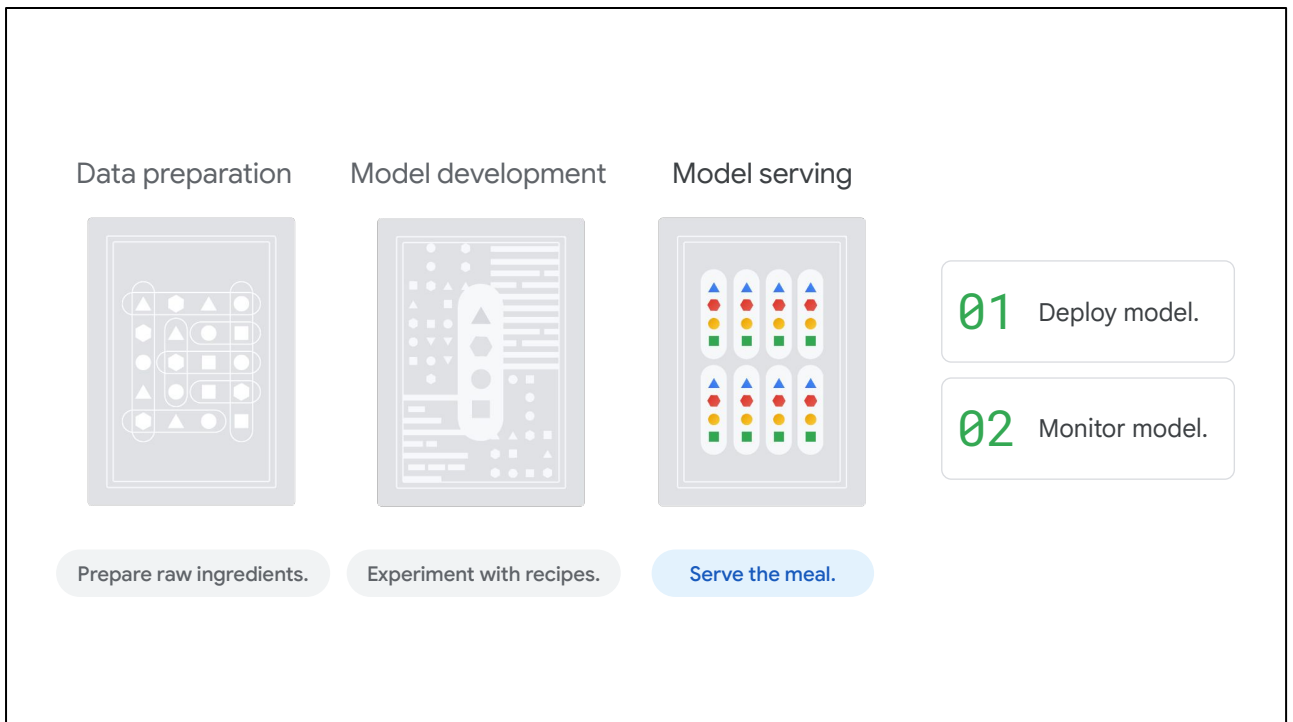


Agenda



- | | |
|----|---------------------------------|
| 01 | ML workflow |
| 02 | Data preparation |
| 03 | Model development |
| 04 | Model serving |
| 05 | MLOps and workflow automation |
| 06 | Lab: AutoML |
| 07 | How a machine learns (optional) |
| 08 | Summary |

Let's focus on the third stage of the ML workflow, model serving.



The recipes are ready, and now it's time to serve the meal.

This represents the final stage of the machine learning workflow, **model serving**.

Model serving consists of two steps:

- First, **model deployment**, which you can compare to serving a meal to a hungry customer
- Second, **model monitoring**, which is like checking with the waitstaff to ensure that the restaurant is operating efficiently.

It's important to note that model management exists throughout this whole workflow to manage the underlying machine learning infrastructure. This lets data scientists focus on what to do, and not on how to do it.

01

Model
deployment



Endpoint

- Delivers real-time predictions.
- Operates via a deployed endpoint.
- Is ideal for low-latency, on-demand results.

Batch prediction

- Makes batch predictions.
- Does not deploy to an endpoint.
- Is used when no immediate response is required.

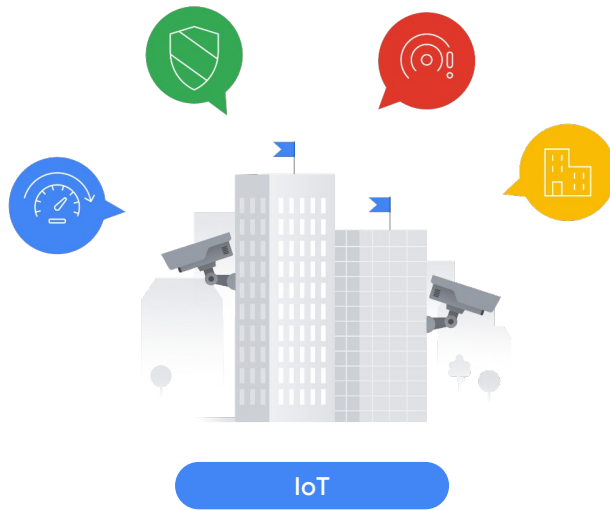
Let's start with **model deployment**, which is the exciting time when the model is implemented and ready to serve.

You have two primary options:

- Deploy the model to an **endpoint** for real-time predictions (or often called online predictions). This option is best when immediate results with low latency are needed, such as making instant recommendations based on a user's browsing habits whenever they're online. A model must be deployed to an endpoint before it can be used to serve real-time predictions.
- Request the prediction job directly from the model resource for **batch prediction**. This option is best when no immediate response is required. For example, sending out marketing campaigns every other week based on the user's recent purchasing behavior and what's currently popular on the market. Batch prediction does not require deploying the model to an endpoint.

You can deploy a model either using the UI on Vertex AI or using code by calling APIs.

You'll practice building an endpoint later in the lab.



Beyond making predictions in the cloud, deploying the model “off-cloud” with edge computing is also possible.

This approach is generally adopted when the model needs to be deployed in a specific environment to mitigate latency, ensure privacy, or enable offline functionality

For instance, consider an IoT (Internet of Things) application like object detection that utilizes a camera feed in a manufacturing plant. In such use case, the added latency of relying on the cloud can be impractical.

02

Model
monitoring



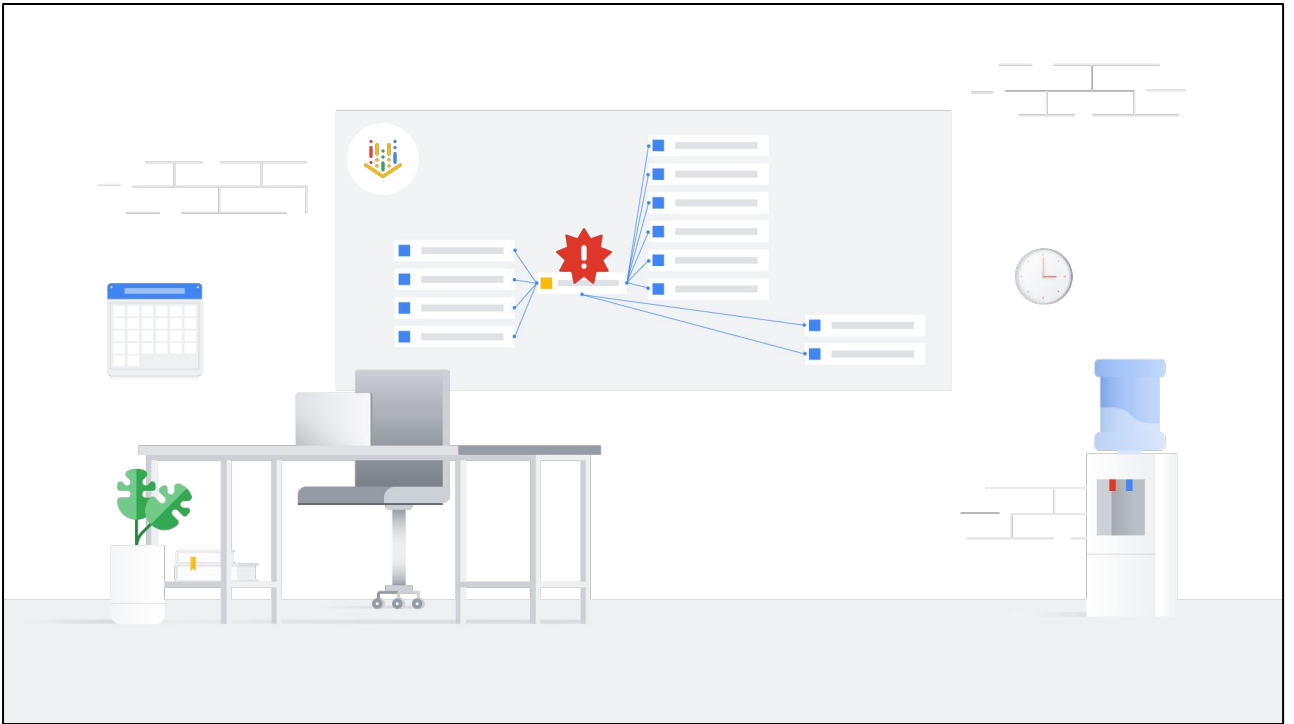
Vertex AI Pipelines

- ✓ Automate
- ✓ Monitor
- ✓ Govern

Once the model is deployed and begins making predictions or generating contents, it is important to *monitor* its performance.

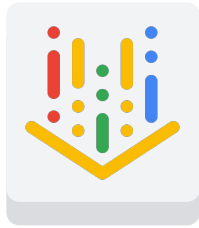
The backbone of automating ML workflow on Vertex AI is a tool kit called *Vertex AI Pipelines*.

It automates, monitors, and governs machine learning systems by orchestrating the workflow in a serverless manner.



Imagine you're in a production control room and Vertex AI Pipelines is displaying the production data on-screen.

If something goes wrong, it automatically triggers and displays a warning based on a predefined threshold.



Vertex AI Workbench
Colab Enterprise



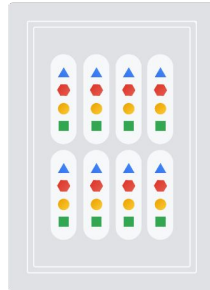
Define your own pipeline
with prebuilt pipeline
components.

With Vertex AI Workbench and Colab Enterprise, which are notebook tools, you can define your own pipeline using SDKs. You can do this with prebuilt pipeline components, which means that you primarily need to specify how the pipeline is put together using components as building blocks. You'll explore more details about Vertex AI Pipelines in the next lesson.

Model serving

01 Deploy model.

02 Monitor model.



And it's with these final two steps, model deployment and model monitoring, that you complete the exploration of the machine learning workflow.

The restaurant is open and operating smoothly. Bon appetit!

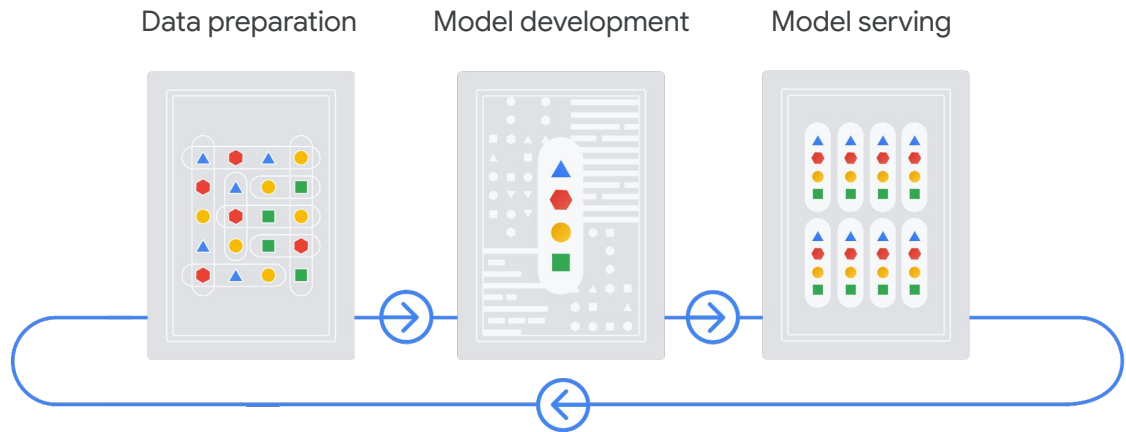


Agenda



- | | |
|----|---------------------------------|
| 01 | ML workflow |
| 02 | Data preparation |
| 03 | Model development |
| 04 | Model serving |
| 05 | MLOps and workflow automation |
| 06 | Lab: AutoML |
| 07 | How a machine learns (optional) |
| 08 | Summary |

In this lesson, you explore an advanced topic: MLOps and workflow automation.



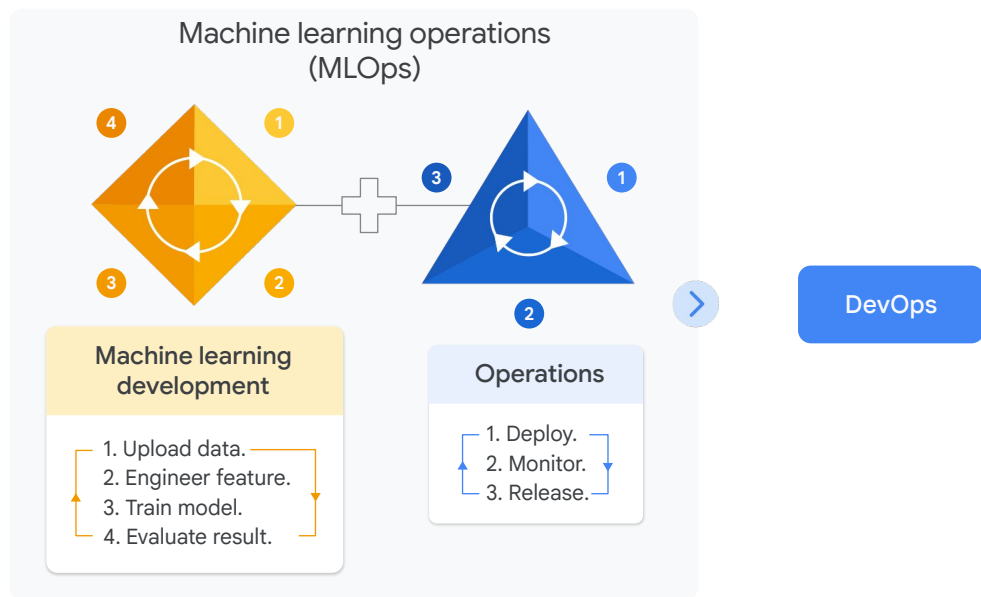
You learned from the previous lessons to build an ML model through three main stages: data preparation, model development, and model serving.



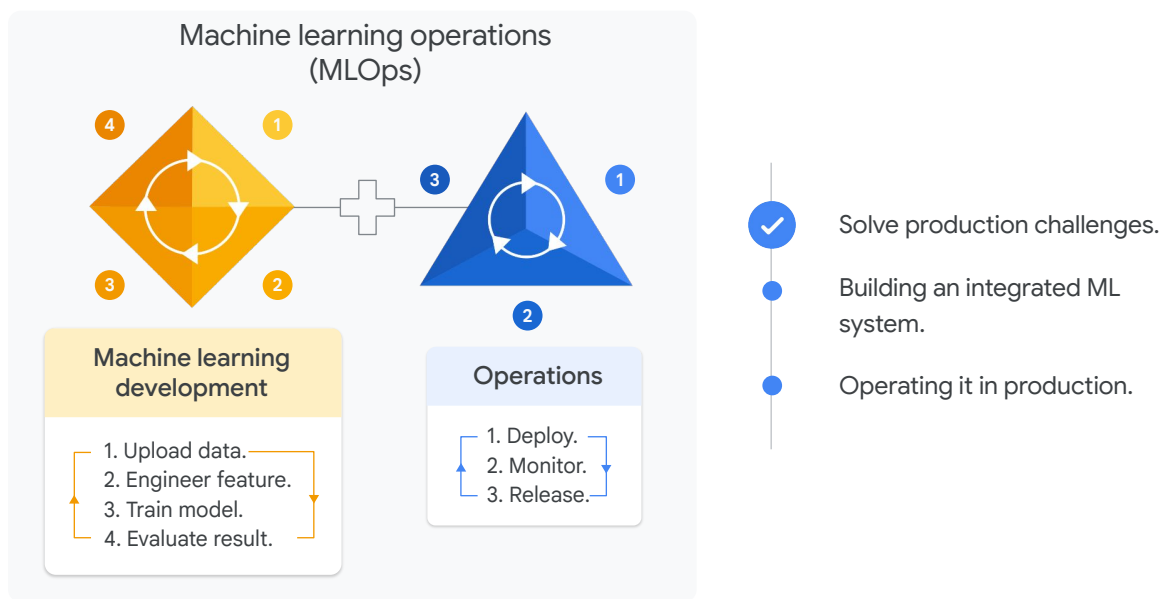
You have two approaches to build an end-to-end workflow. One is codeless through Google Cloud console, like AutoML on Vertex AI.

But what if you want to automate this workflow to achieve continuous integration, training, and delivery?

Here comes the other option: to *code* a pipeline that automates the ML workflow.

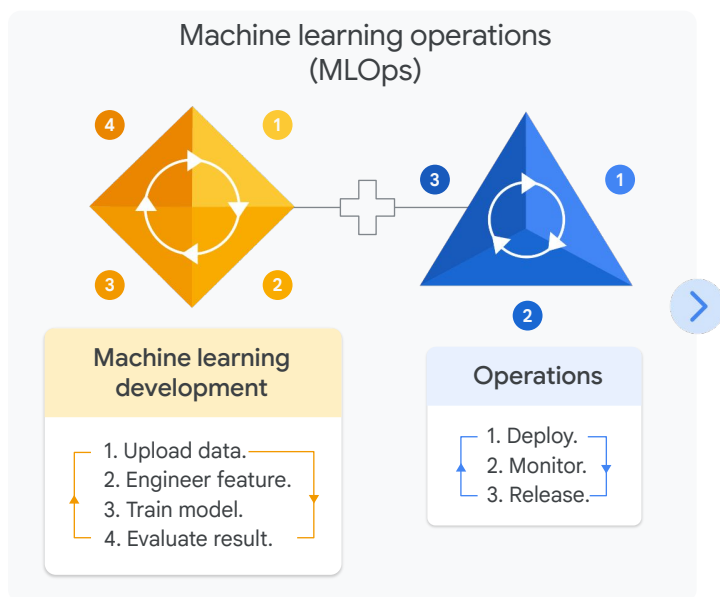


Machine learning operations, or MLOps, play a big role. MLOps combines machine learning development with operations and applies similar principles from DevOps (or development operations) to machine learning models.



MLOps aims to solve production challenges related to machine learning.

In this case, this refers to building an integrated machine learning system and operating it in production. These are considered to be some of the biggest pain points by the ML practitioners' community because *both data and code* are constantly evolving in machine learning.

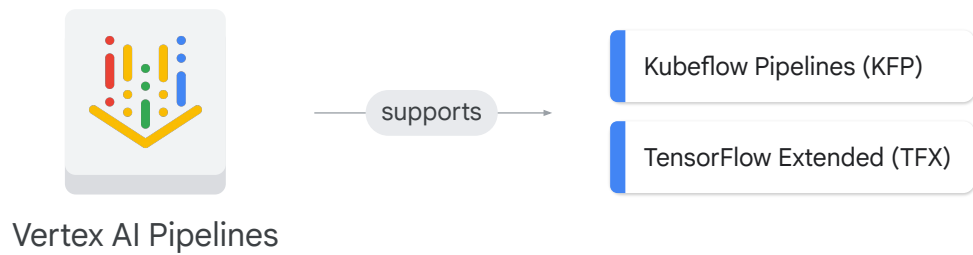


Practicing MLOps

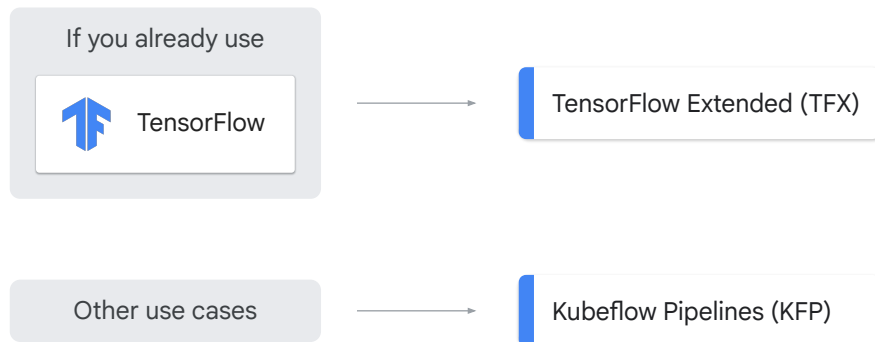
Automating and monitoring each step of the ML system construction.

- Continuous integration (CI)
- Continuous training (CT)
- Continuous delivery (CD)

Practicing MLOps means automating and monitoring each step of the ML system construction to enable continuous integration, training, and delivery.

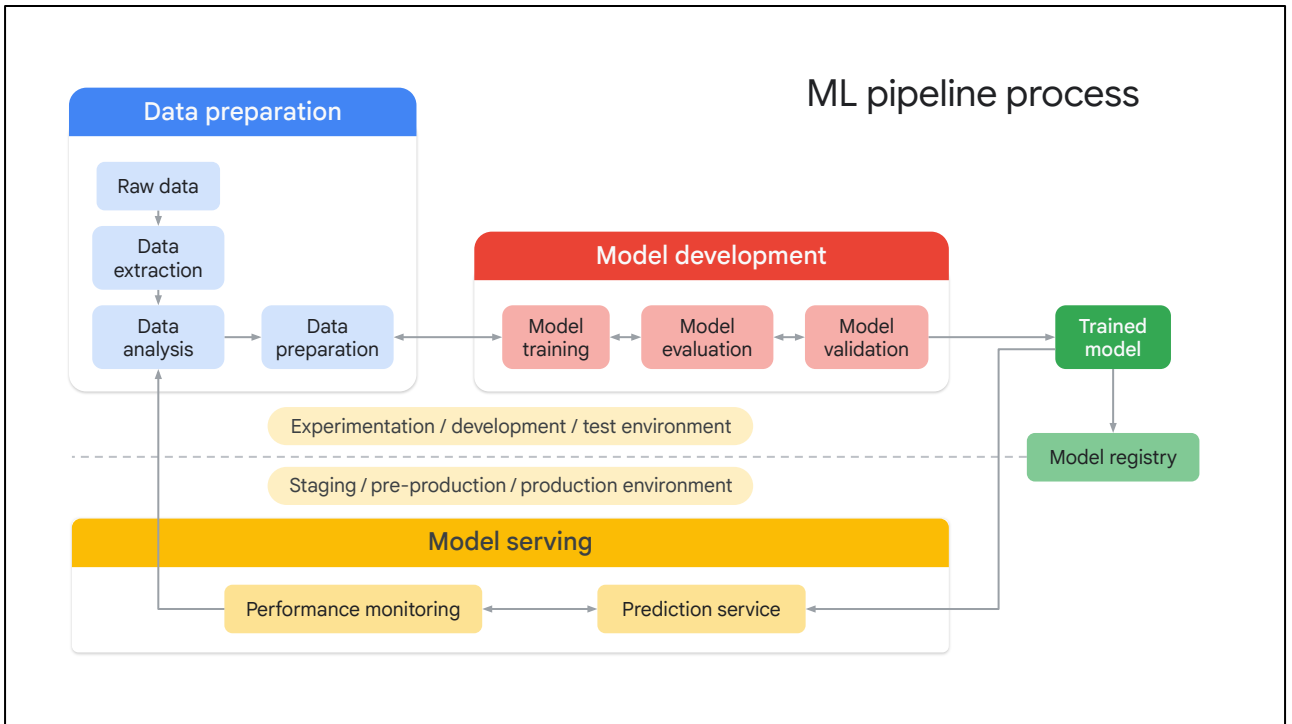


The backbone of MLOps on Vertex AI is a tool kit called *Vertex AI Pipelines*. This tool kit supports both Kubeflow Pipelines, or KFP, and TensorFlow Extended, or TFX.



If you already use TensorFlow to build ML models that process terabytes of structured data, it makes sense to use TFX and turn that code into an ML pipeline.

Otherwise, KFP can be a good alternative. Learn more about how to [choose between the Kubeflow Pipelines SDK and TFX](#) from the reading list.



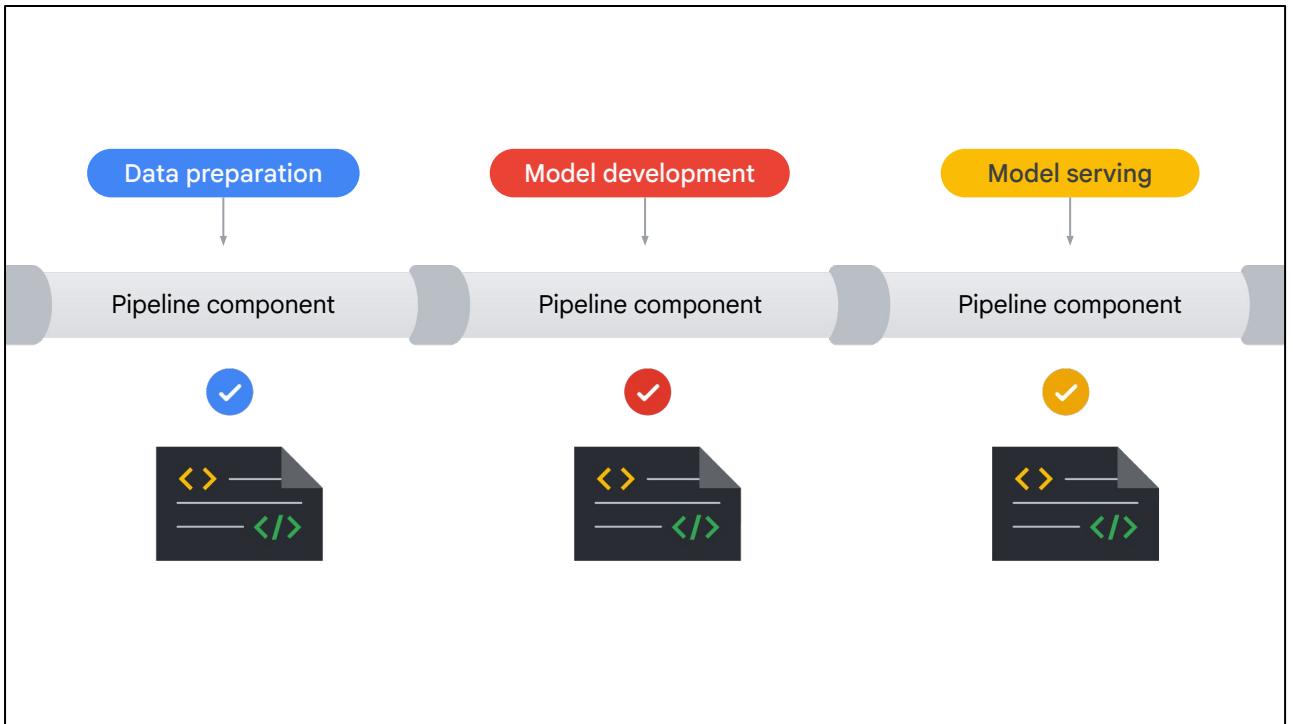
An ML pipeline contains a series of processes and runs in two different environments:

- First is the experimentation, **development**, and test environment.
- Second is the staging, pre-production, and **production** environment.

In the development environment, you start from **data preparation** which includes data extraction, analysis, and preparation, to **model development** like training, evaluation, and validation.

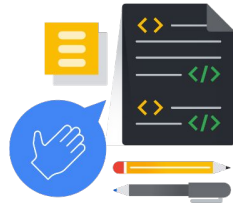
The result is a trained model that can be entered in model registry.

Once the model is trained, the pipeline moves to the **staging and production environment**, where you **serve the model**, which includes prediction and monitoring.



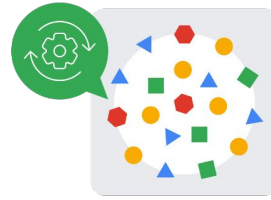
Each of these processes can be a **pipeline component**, which is a **self-contained set of code** that performs one task of a workflow.

You can think of a component as a function, which is a building block of a pipeline.



Custom component

Accomplish a specific task to tailor to your ML workflow.



Pre-built component

Reuse or customize standard components to suit your needs.

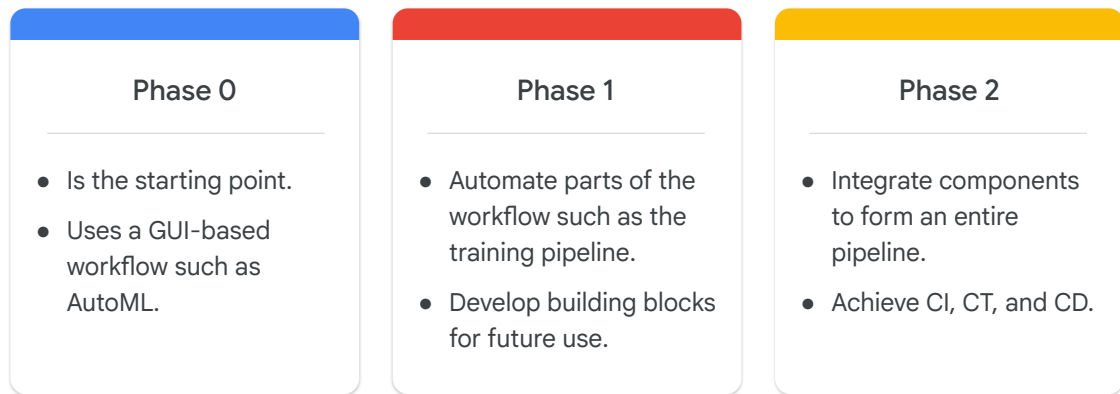
You can either build a **custom component** on your own or leverage the **pre-built components** provided by Google.

If you want to accomplish a specific task to tailor your ML workflow, such as determining a special threshold for model deployment, you may need to code a **custom component**.

Before doing so, check the **pre-built components** offered by Google Cloud. You may find a pipeline component to reuse or customize to suit your needs.

Learn more about [using Google Cloud pipeline components in your pipeline](#) in the reading list.

Phases of ML automation

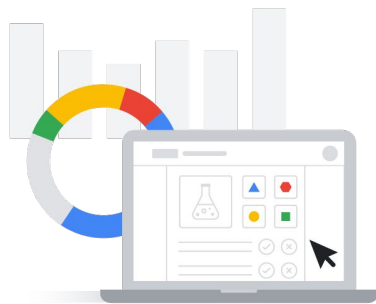


All these components are like pieces of an ML pipeline. You need to assemble them together to automate the entire ML workflow.

Organizations often implement ML automation in three phases:

- **Phase zero** is the starting point, where you have not configured any MLOps. You typically use the graphical user interface, or GUI, based workflow such as AutoML for training, deployment, and serving. Phase zero is critical because it helps you build an end-to-end workflow manually before you automate it.
- In **phase one**, you start automating your ML workflow by building components using the Vertex AI Pipelines SDKs. An example of a component would be the training pipeline. It is in this phase that you develop the building blocks for future use.
- In **phase two**, you integrate the separate components to form an entire workflow and to achieve CI, CT, and CD.

Pipeline example



Build a pipeline to train, evaluate, and deploy an AutoML model that classifies beans into 1 of 7 types based on their characteristics.



Let's look at an example. Assume you want to build a pipeline to train, evaluate, and deploy an AutoML model that classifies beans into 1 of 7 types based on their characteristics.

You have two main steps: build a pipeline and then run it.

Build a pipeline

01 Plan pipeline components.

02 Build custom components.

03 Add prebuilt components.

Run a pipeline

01 Compile the pipeline.

02 Define and run the job.

Design a series of components, which can be a combination of custom and pre-built components.

```
@component()
def classification_model_eval_metrics():

@pipeline()
def pipeline():
    Tdataset_create_op =
    gcc_aip.TabularDatasetCreateOp()
    Training_op =
    gcc_aip.AutoMLTabularTrainingJobRunOp()
    model_eval_task =
    classification_model_eval_metrics()
    endpoint_op = gcc_aip.EndpointCreateOp()
    gcc_aip.ModelDeployOp()
```

To build a pipeline:

- You **first** plan it as a series of components, which can be a combination of custom and pre-built. To promote reusability, each component should have a single responsibility.
- **Second**, you build any custom components that are needed. For example, you create a component called `classification_model_eval_metrics`. You use this component to compare the evaluation metrics to a threshold after the model is trained, and determine whether the model should be deployed. If the model performs well, you deploy it. Otherwise, you retrain the model.
- **Third**, you assemble the pipeline by adding pre-built components, for example:
 - `TabularDatasetCreateOp` creates a tabular dataset in Vertex AI given a dataset source either in Cloud Storage or BigQuery.
 - `AutoMLTabularTrainingJobRunOp` kicks off an AutoML training job for a tabular dataset.
 - `EndpointCreateOp` creates an endpoint in Vertex AI.
 - And `ModelDeployOp` deploys a given model to an endpoint in Vertex AI.

You also include a custom component from the previous step, **`classification_model_eval_metrics`**, to compare the performance of the trained model to a threshold.

Build a pipeline

01 Plan pipeline components.

02 Build custom components.

03 Add prebuilt components.

Run a pipeline

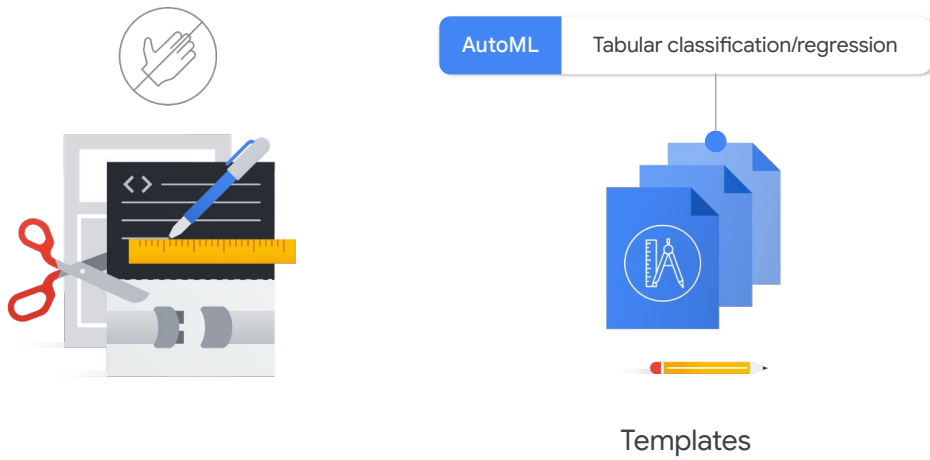
01 Compile the pipeline.

02 Define and run the job.

```
ml_pipeline_job = aiplatform.PipelineJob(  
    display_name="automl-tab-beans-training",  
  
    template_path="tab_classif_pipeline.json",  
    pipeline_root=PIPELINE_ROOT,  
    parameter_values={"project": PROJECT_ID,  
        "display_name": DISPLAY_NAME},  
    enable_caching=True  
)  
  
ml_pipeline_job.submit()
```

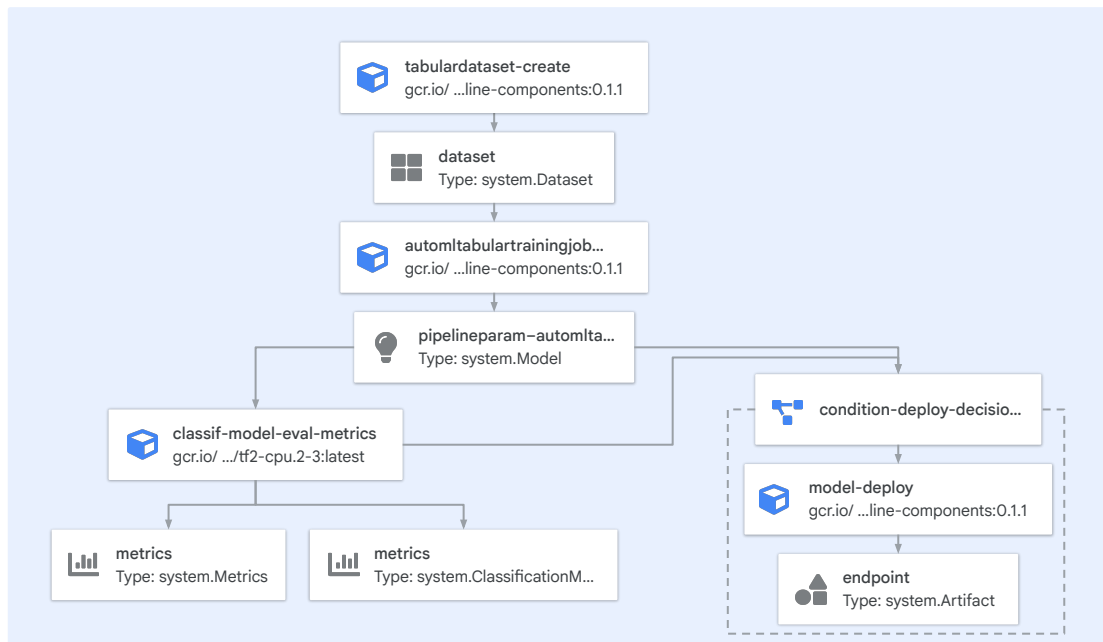
After the pipeline is built, you must compile and run it.

- First, you compile it using the `compiler.Compiler().compile()` or `compile()` commands.
- And then, you define and run the pipeline job.



The good news is you don't have to create a pipeline from the beginning.

Vertex AI provides a few templates, like the one for classification/regression of tabular data with AutoML, to help you start your journey.



Now you have an automated pipeline to train, evaluate, and deploy an ML model. This pipeline will check the performance of the model constantly and decide whether it should be deployed or retained without your intervention.

The nice thing is that Google Cloud also visualizes the pipeline based on the coding, with which you can easily check the components and the corresponding artifacts.

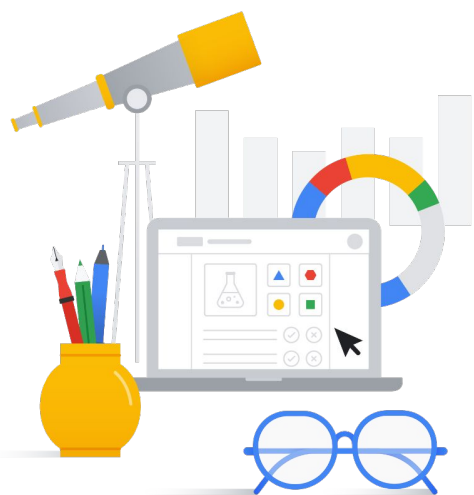
This example demonstrated the overall process to build a pipeline.

Learn more about pipelines:



[Demo: Introduction to Vertex AI Pipelines](#)

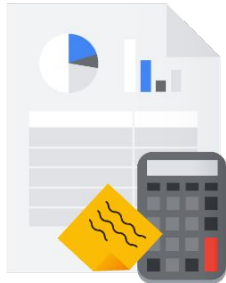
To know more about pipeline details, please practice with the coding example in the demo: Introduction to Vertex AI Pipelines, which is available in the reading list for this course.



Hands-on lab:

Vertex AI: Predicting Loan
Risk with AutoML

Let's put what you've just learned into practice with a hands-on lab.



Use AutoML to build a machine learning model to predict loan risk.

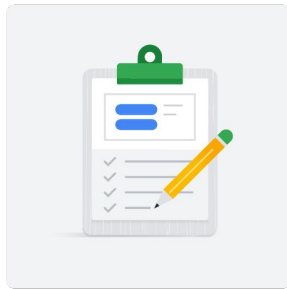
> Loans from a financial institution

> 2,050 data points

AutoML requires at least 1,000 data points

In this lab, you'll use AutoML, a no-code tool, to build a machine learning model to predict loan risk.

The dataset used in the lab relates to loans from a financial institution and has 2,050 data points. AutoML requires at least 1,000 data points in a dataset.



AutoML lab

Practice working through the three phases of the ML workflow:

- 1 Data preparation
- 2 Model development
- 3 Model serving

The goal is to practice working through the three phases of the machine learning workflow: data preparation, model development, and model serving.

True label	Predicted label	
	0 (repay)	1 (not repay)
0 (repay)	100%	—
1 (not repay)	13%	87%

Before you start the lab, let's explain the details about model evaluation so that you can interpret the training results.

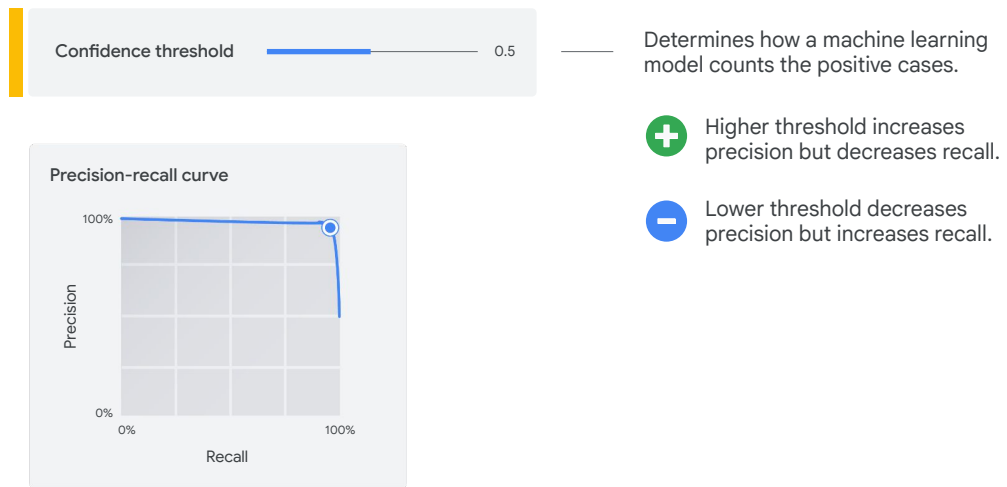
Let's start with the confusion matrix. You'll get a similar result like this in the lab. Pause for a second and try to interpret this matrix yourself. What does it tell you?

Confusion matrix result

		Predicted label	
		0	1
True labels	0	100% $\frac{TP}{TP+FN}$	— $\frac{FN}{TP+FN}$
	1	13% $\frac{FP}{FP+TN}$	87% $\frac{TN}{FP+TN}$

- **100% true positive rate:** The positive class here is 'repay' (0), the desired business outcome. This 100% means that the model is *perfect* at identifying everyone who will actually repay their loan. It never misses a good, safe customer. That's a fantastic outcome for maximizing business opportunities. Note that the True Positive Rate equals True Positives divided by the sum of True Positives and False Negatives. If the terms sound unfamiliar, please refer to the previous example, where you learned about the confusion matrix.
- **87% true negative rate:** The negative class is 'not repay' (1), the high-risk outcome. This 87% means that the model correctly identifies **87% of all the people who are actually defaulters**. This is a strong result for risk management, as the bank successfully catches and rejects most of the high-risk applications. Note that the True Negative Rate equals True Negatives divided by the sum of False Positives and True Negatives.
- **13% false positive rate:** A 'False Positive' is the most expensive mistake: the model predicts the customer is safe ('repay' or 0) when they would have actually defaulted ('not repay' or 1). This means that **13% of people who will actually default on their loan are mistakenly approved**. This directly leads to financial loss for the bank. Note that the False Positive Rate equals False Positives divided by the sum of False Positives and True Negatives.
- Finally, **0% false negative rate:** A 'False Negative' is the error where the model predicts risk ('not repay' or 1) when the customer would have been safe

- ('repay' or 0). Since this rate is 0%, the model **never incorrectly rejects a customer who is actually a safe borrower**. The bank avoids all lost business opportunity from turning away good customers. This perfect rate for customer acceptance is a core strength, despite the trade-off of approving some bad loans seen in the False Positive rate.



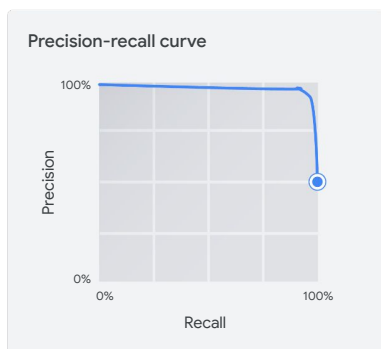
Let's look at the precision-recall curve you will encounter in the upcoming AutoML lab.

The confidence threshold determines how a machine learning model counts the positive cases.

- A higher threshold increases the precision but decreases recall.
- A lower threshold decreases the precision, but increases recall.

Confidence threshold 0

Determines how a machine learning model counts the positive cases.



⊕ Higher threshold increases precision but decreases recall.

⊖ Lower threshold decreases precision but increases recall.

0 Zero threshold produces the highest recall of 100% and the lowest precision of 50%.

The model predicts that 100% of loan applicants will be able to repay a loan they take out. However, actually only 50% of people could repay the loan.

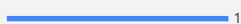
Moving the confidence threshold to zero produces the highest recall of 100% and the lowest precision of 50%.

So, what does that mean?

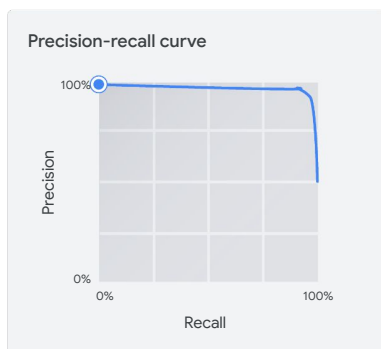
That means the model *predicts* that 100% of loan applicants will be able to repay a loan they take out. However, actually only 50% of people were able to repay the loan.

Using this threshold to identify the default cases in this example can be risky, because it means that you can only get half of the loan investment back.

Confidence threshold



Determines how a machine learning model counts the positive cases.



Higher threshold increases precision but decreases recall.



Lower threshold decreases precision but increases recall.

1

A threshold of 1 produces the highest precision of 100% with the lowest recall of 1%.

Of all the people who were predicted to repay the loan, 100% of them actually did. However, you rejected 99% of loan applicants by only offering loans to 1% of them.

Now let's view the other extreme by moving the threshold to 1. This will produce the highest precision of 100% with the lowest recall of 1%.

What does *this* mean?

It means that, of all the people who were predicted to repay the loan, 100% of them actually did. However, you rejected 99% of loan applicants by only offering loans to 1% of them. That's a pretty big business loss for your company.

These are both extreme examples, but it's important that you always try to set an appropriate threshold for your model.

Now that we've made a review, let's start the lab.

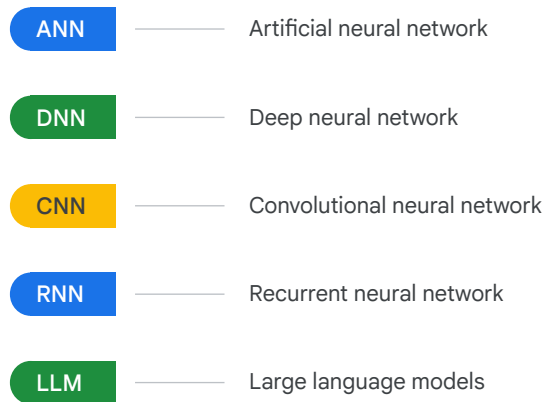


Agenda



- | | |
|----|---------------------------------|
| 01 | ML workflow |
| 02 | Data preparation |
| 03 | Model development |
| 04 | Model serving |
| 05 | MLOps and workflow automation |
| 06 | Lab: AutoML |
| 07 | How a machine learns (optional) |
| 08 | Summary |

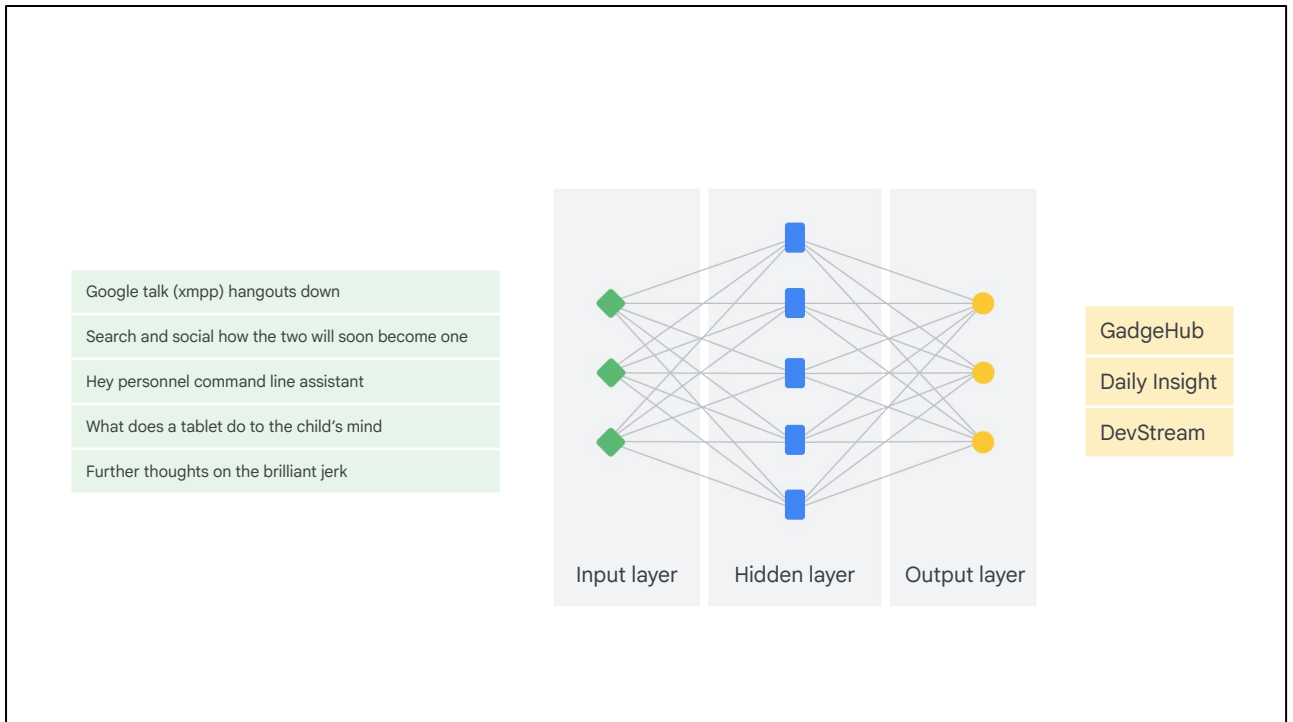
To understand machine learning, you must first understand how neural networks learn. This includes exploring this learning process and the terms associated with it. If you are already familiar with the ML theories and terminologies, feel free to skip this lesson.



How do machines learn? And how do they assess their learning?

Before you dive into building an ML model, let's take a look at how a neural network learns.

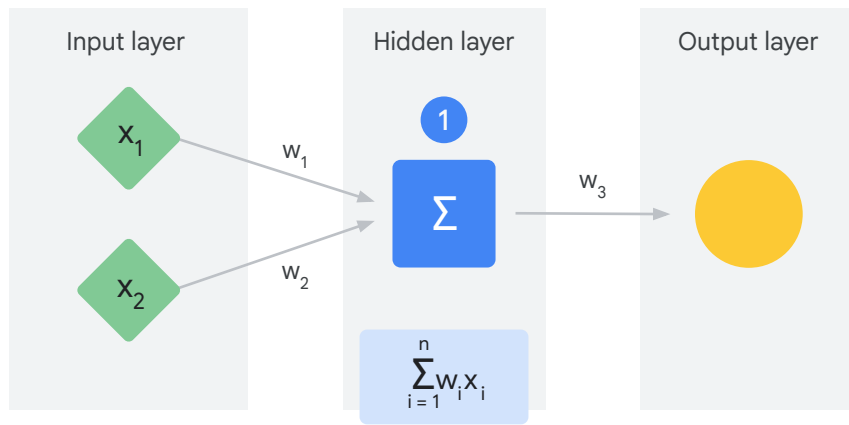
You may already know about various neural networks, such as deep neural networks (or DNN), convolutional neural networks (or CNN), recurrent neural networks (or RNN), and more recently large language models (LLMs). These networks are used to solve different problems. All of these models stem from the most basic: artificial neural network (or ANN). ANNs are also referred to as neural networks or shallow neural networks.



Let's focus on ANN to see how a neural network learns. An ANN has three layers: an input layer, a hidden layer, and an output layer. Each node represents a neuron. The lines between neurons stimulate synapses, which is how information is transmitted in a human brain.

For instance, if you input article titles from multiple resources, the neural network can tell you which media outlet or platform the article belongs to, such as GadgeHub, New York Times, and DevStream.

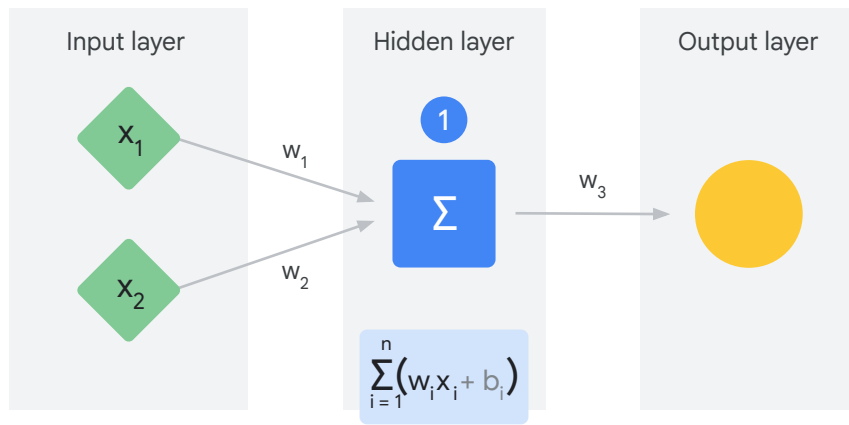
How does an ANN learn from examples and then make predictions? Let's examine how it works in depth.



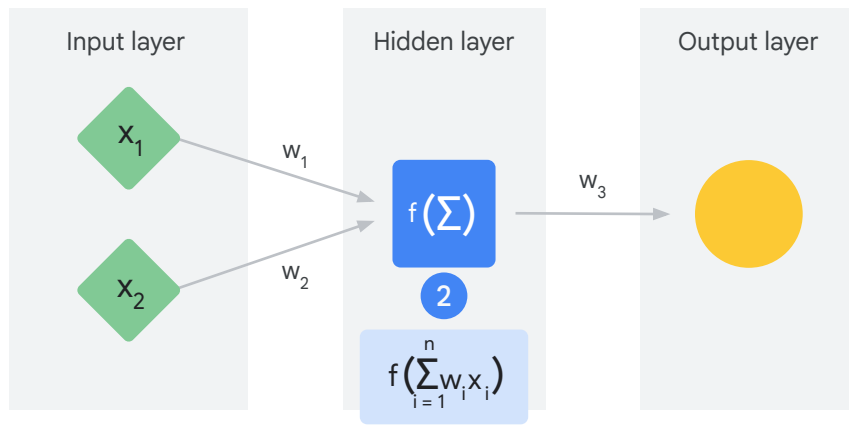
Let's assume you have two input neurons or nodes, one hidden neuron, and one output neuron.

Above the link between neurons are **weights**. The weights retain information that a neural network learned through the training process and are the **mysteries** that a neural network aims to discover.

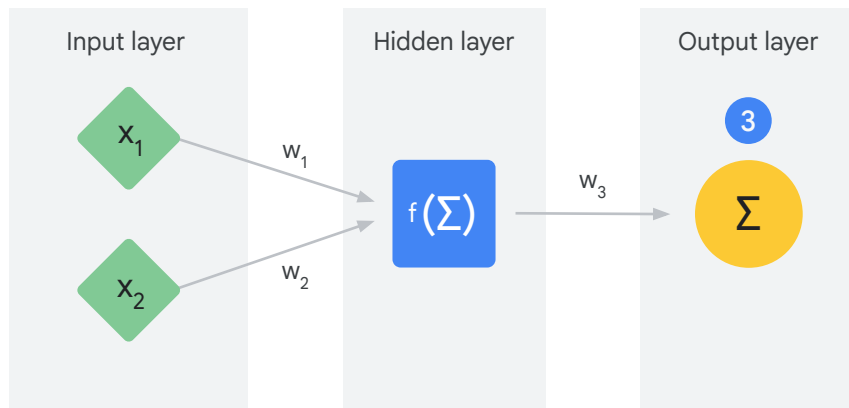
The first step is to calculate the weighted sum. This is done by multiplying each input value by its corresponding weight, and then summing the products.



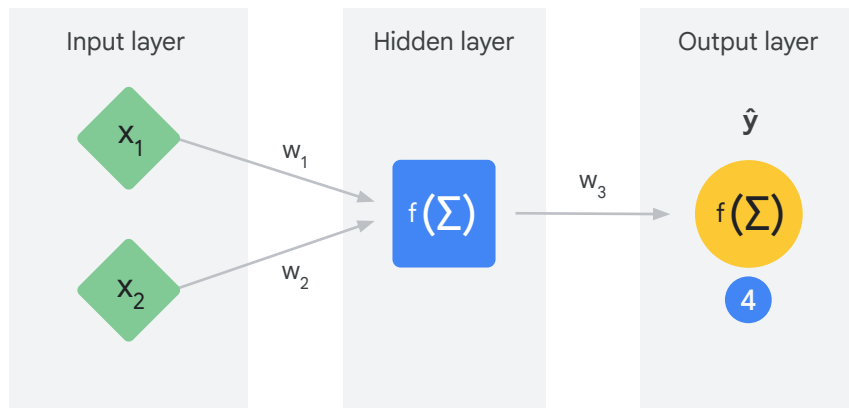
It normally includes a *bias* component b_i . However, to focus on the core idea, ignore it for now.



The second step is to apply an *activation function* to the weighted sum. What is an activation function, and why do you need it? Let's pause your curiosity for just a moment and get back to that soon.



In the third step, the weighted sum is calculated for the output layer, assuming multiple neurons in the hidden layers.



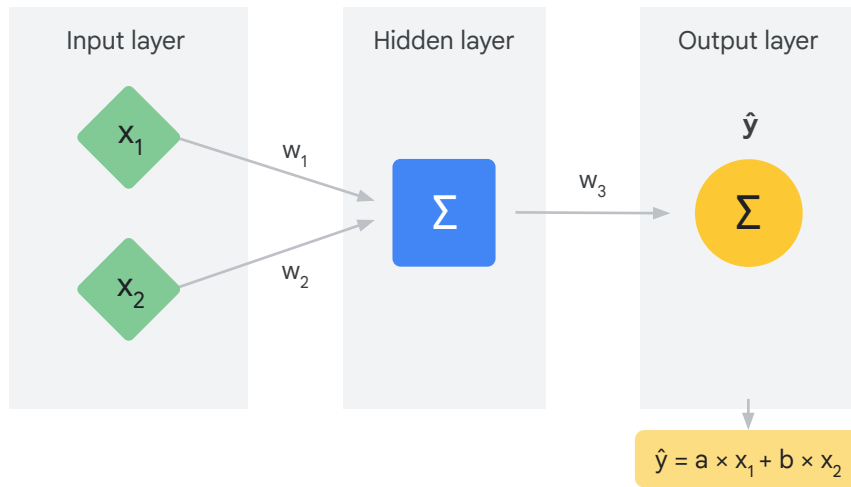
The fourth step is to apply an **activation function** to the weighted sum. This activation function can be different from the one applied to the hidden layers.

The result is the **predicted y** , which consists of the output layer. You use **$\hat{y}$** to represent the **predicted result** and **y** as the **actual result**.

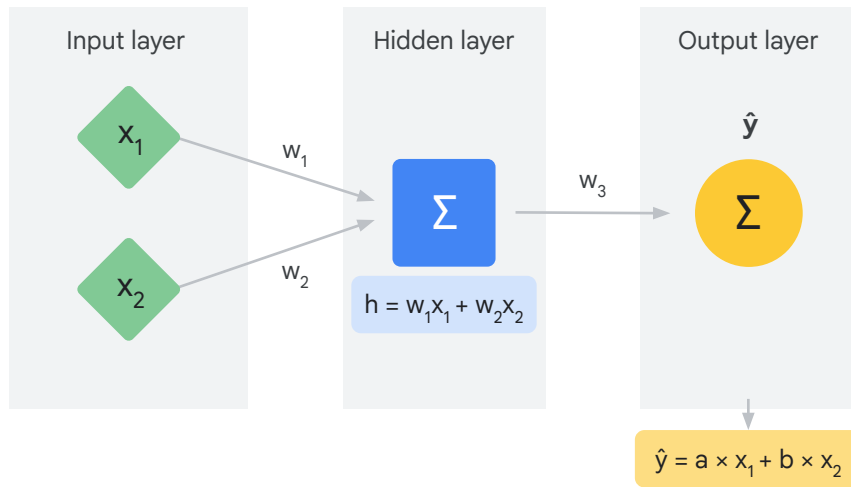
An activation function is
used to **prevent linearity**.

Now let's get back to activation functions. What does an activation function do?

Well, an activation function is used to prevent linearity or *add* non-linearity. What does that mean? Think about a neural network.

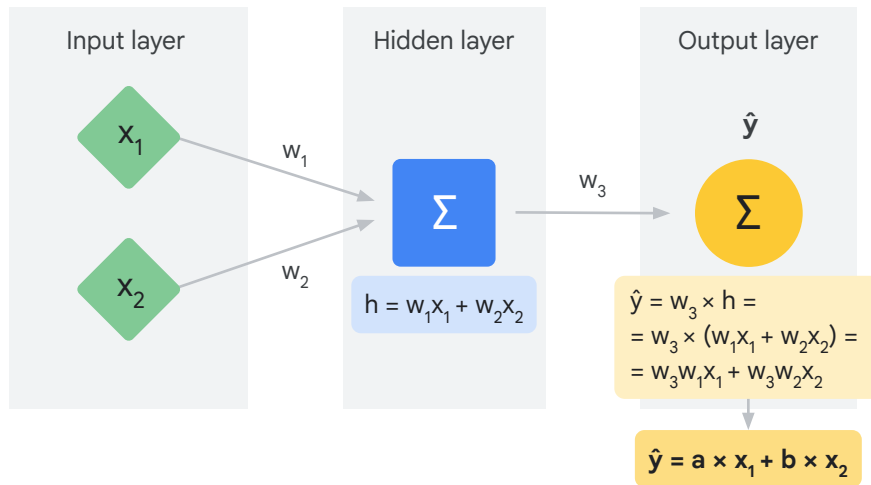


Without activation functions, the predicted result y will always be a linear function of the input x , regardless of the number of layers between input and output.



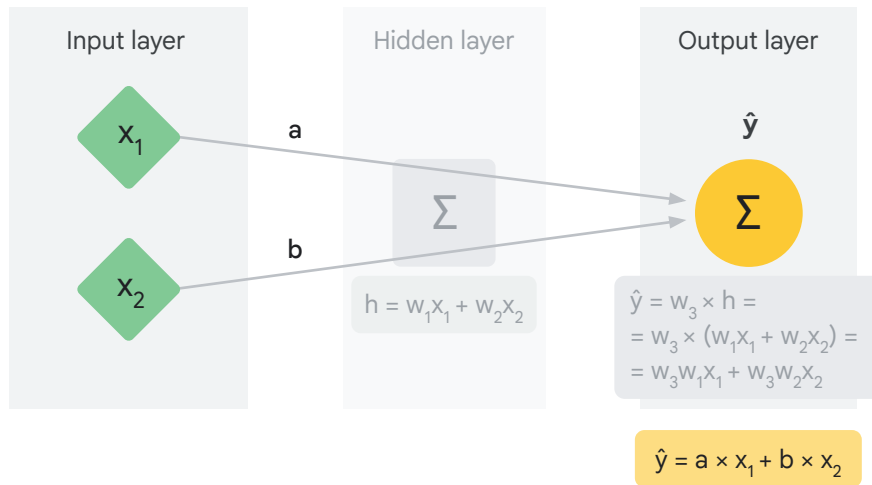
Let's walk through this for clarity. Without the activation function, the value of the hidden layer h equals a total of w_1 times x_1 and w_2 times x_2 .

Please note that, to make this illustration easy, we ignored the bias component b , which you often see in other ML materials.



The output $\hat{y}$ therefore equals to w_3 times h , and eventually equals to a total of constant number a times x_1 and a constant number b times x_2 .

In other words, the output $\hat{Y}$ is a linear combination of the input X .

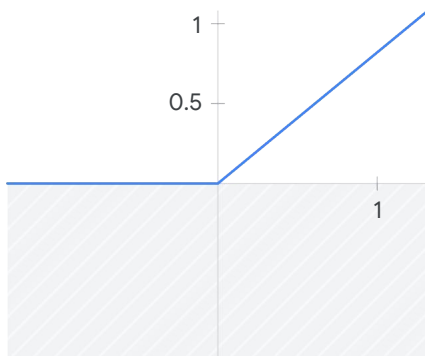


If y is a linear function of x , you don't need all the hidden layers, but only one input and one output. You might already know that linear models do not perform well when handling comprehensive problems.

Activation functions **convert**
a linear network to a **non-linear** one.

That's why you must use activation functions to convert a linear network to a non-linear one.

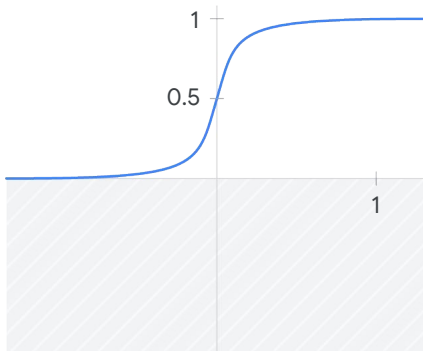
Rectified linear unit (ReLU)



$$f(x) = \begin{cases} 0 & x < 0 \\ x & x \geq 0 \end{cases}$$

What are the widely used activation functions? You can use the rectified linear unit (or ReLU) function, which turns an input value to zero if it's negative, or keeps the original value if it's positive.

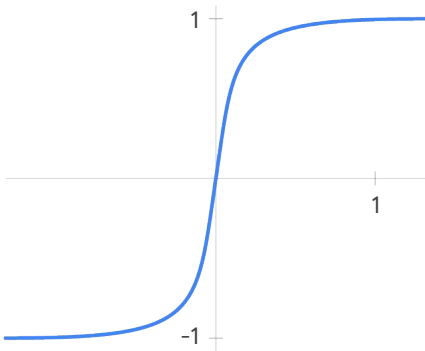
Sigmoid



$$f(x) = \frac{1}{1 + e^{-x}}$$

You can use the sigmoid function, which turns the input to a value between 0 and 1.

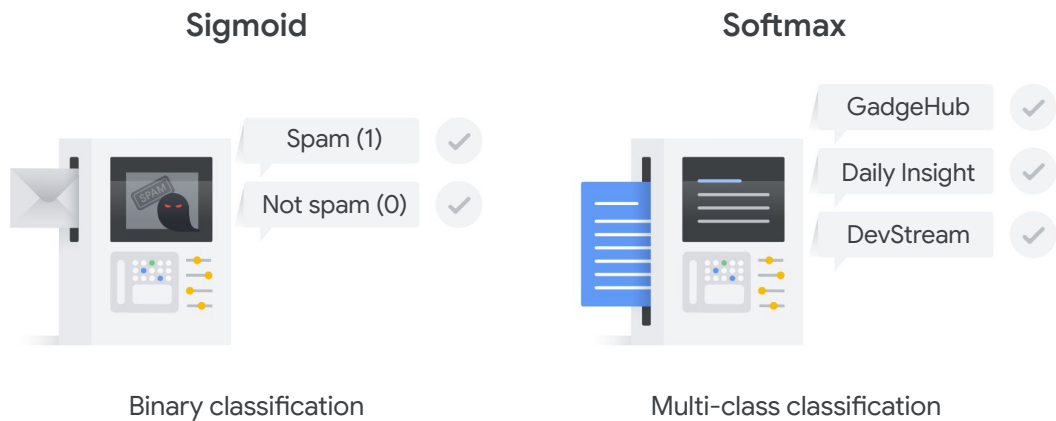
Hyperbolic tangent (tanh)



$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

And hyperbolic tangent (Tanh) function, which shifts the sigmoid curve and generates a value between -1 and +1.

Softmax



Another interesting and important activation function is called **softmax**. Think about sigmoid: it generates a value from zero to one and is used for binary classification in logistic regression models. An example for this would be deciding whether an email is spam.

What if you have multiple categories, such as GadgeHub, Daily Insight, and DevStream? Here you must use softmax, which is the activation function for multi-class classification.

The output of softmax
is a **probability distribution**.

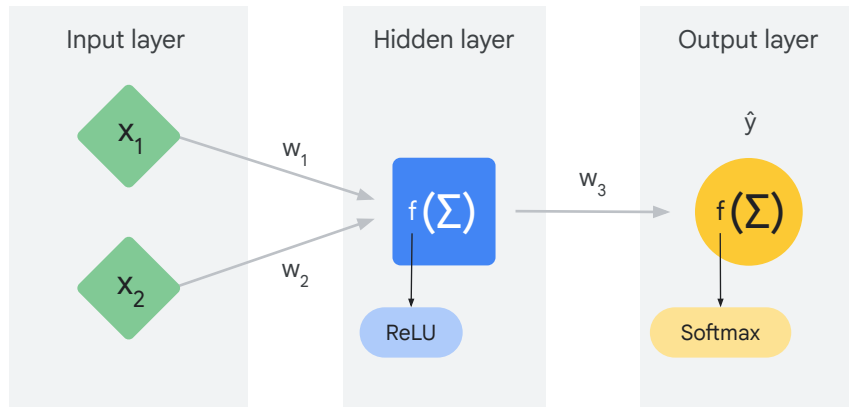
$$f_i(\vec{x}) = \frac{e^{x_i}}{\sum_{j=1}^J e^{x_j}} \quad \text{for } i = 1, \dots, J$$

It maps each output to a $[0,1]$ range in a way that the total adds up to 1. Therefore, the output of softmax is a probability distribution.

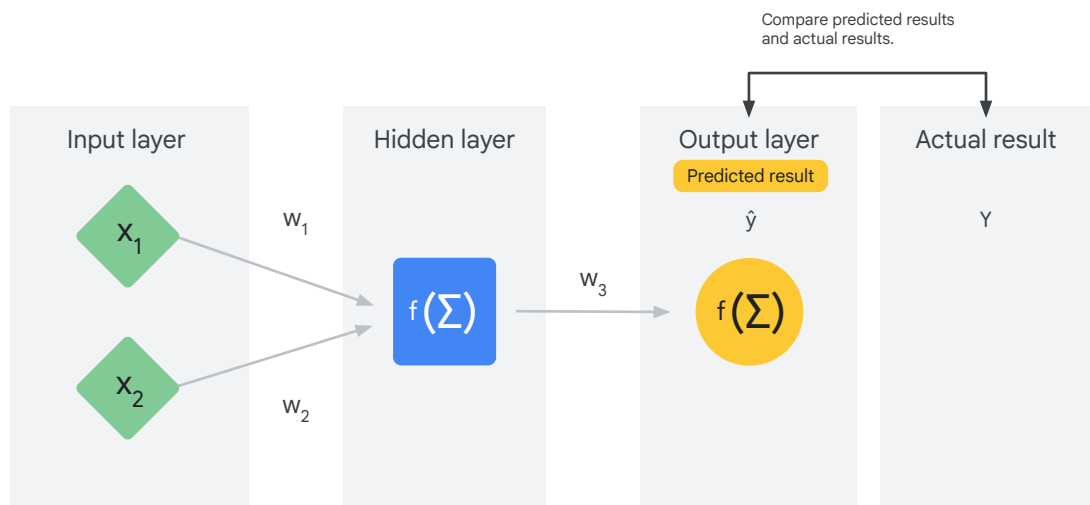
Softmax versus sigmoid

- ✓ Softmax → **multi-class** classification
 - Article title classification: GadgeHub, Daily Insight, and DevStream.
- ✓ Sigmoid → **binary-class** classification
 - Email spam detection: yes or no spam.

Skipping the math, you can conclude that softmax is used for *multi-class classification*, whereas sigmoid is used for *binary-class classification* in logistic regression models.

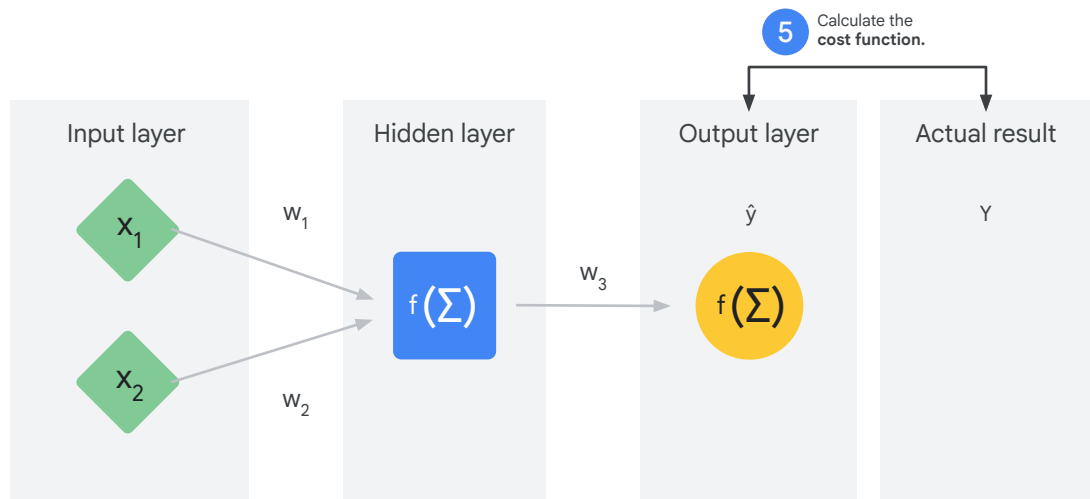


Also note that you don't need to have the same activation function across different layers. For instance, you can have ReLU for hidden layers and softmax for the output layer.



Now that you understand the activation function and get a predicted y , how do you know if the result is correct?

You use an assessment called *loss function* or *cost function* to measure the difference between the predicted y and the actual y . *Loss function* is used to calculate errors for *a single training instance*, whereas *cost function* is used to calculate errors from *the entire training set*.



Therefore, in step five, you calculate the cost function to minimize the difference. If the difference is significant, the neural network knows that it did a bad job in predicting and must go back to learn more and adjust parameters.

Cost functions for regression problems

$$\text{MSE} = \frac{\sum_{i=1}^n (\hat{Y}_i - Y_i)^2}{n}$$

$\hat{Y}_i$ – Predicted value

Y_i – Actual value

n – Size of the training data

MSE: Mean Squared Error

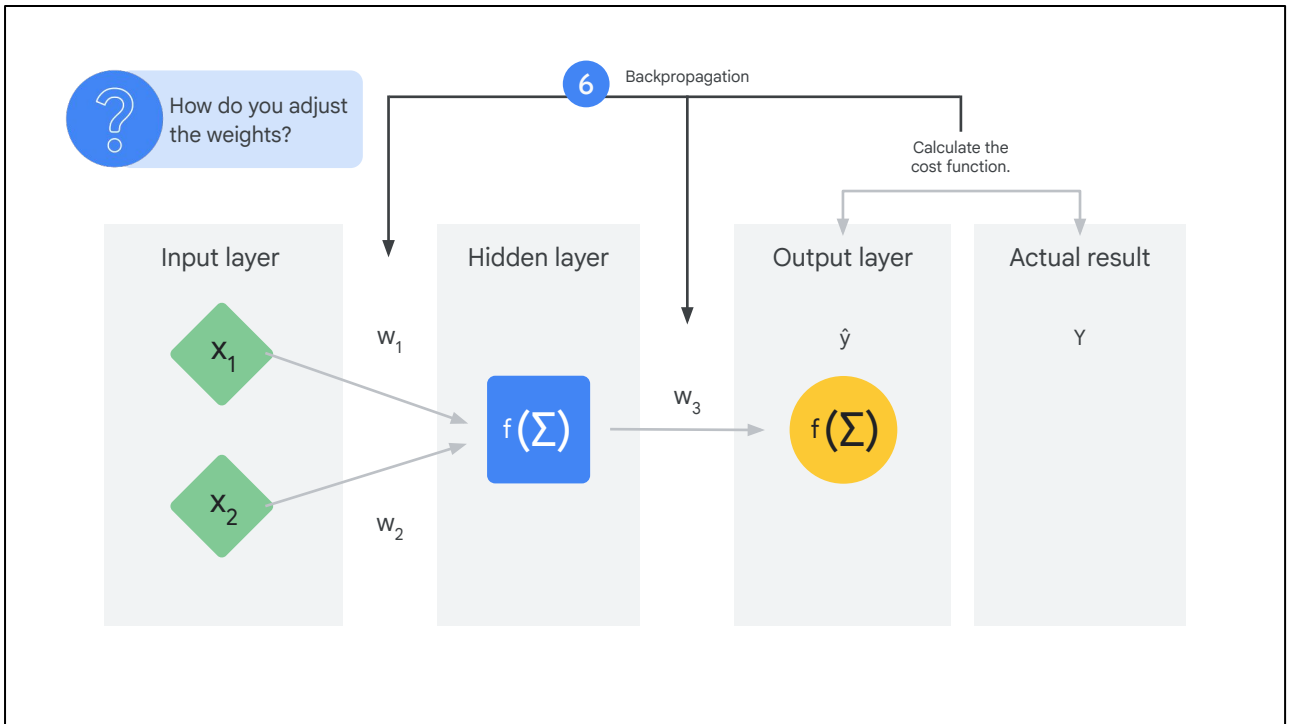
Many different cost functions are used in practice. For regression problems, mean squared error, or MSE, is a common one used in linear regression models. MSE equals the average of the sum of squared differences between $\hat{y}$ and y .

Cost functions for classification problems

Cross-entropy

Calculates the differences between probability distributions.

For classification problems, cross-entropy is typically used to measure the difference between the predicted and actual probability distributions in logistic regression models.



If the difference between the predicted and actual results is significant, you must go back to adjust weights and biases to minimize the cost function. This potential sixth step is called backpropagation.

The challenge now is how to adjust the weights. The solution is slightly complex, but *indeed* the most interesting part of a neural network.

Cost functions

$$\text{MSE} = \frac{\sum_{i=1}^n (\hat{Y}_i - Y_i)^2}{n}$$

$\hat{Y}_i$ – Predicted value

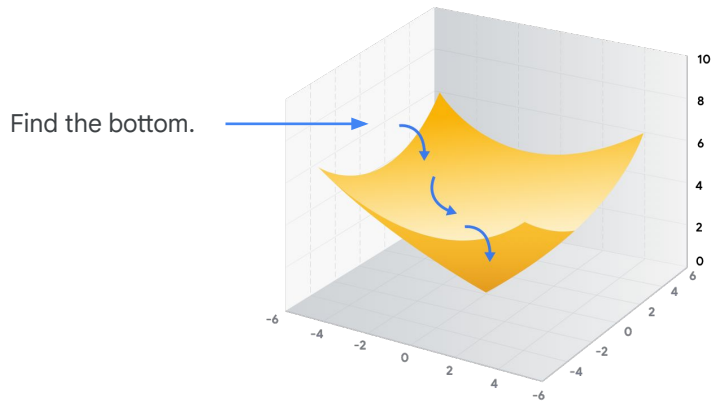
Y_i – Actual value

n – Size of the training data

Search strategy

The idea is to take cost functions and turn them into a search strategy.

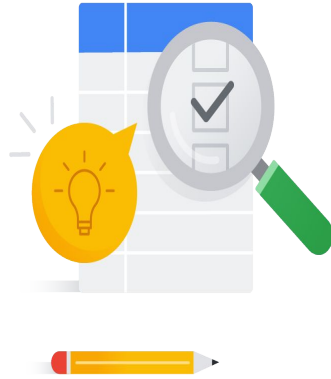
Gradient descent



That's where *gradient descent* comes in. Gradient descent refers to the process of walking down the surface formed by the cost function and finding the bottom.

Finding the bottom

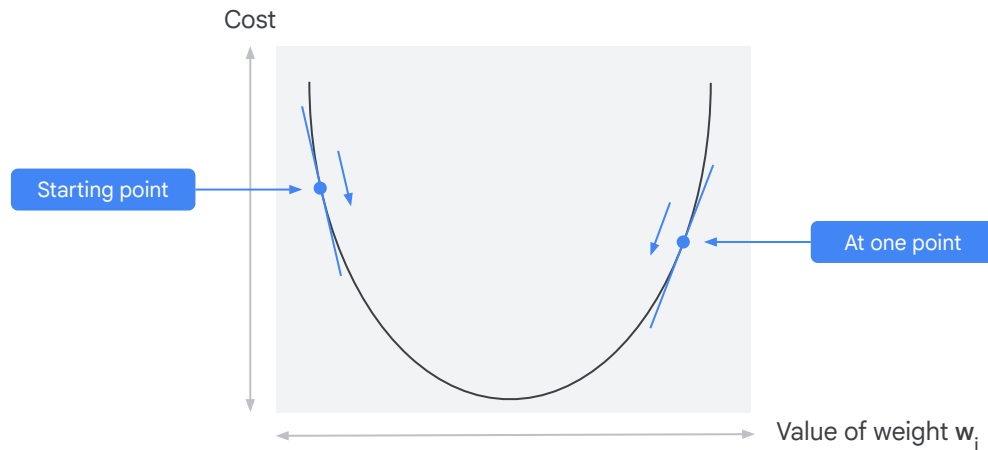
01 Which direction should you take?



It turns out that the problem of finding the bottom can be divided into two different and important questions:

The first one is: Which direction should you take?

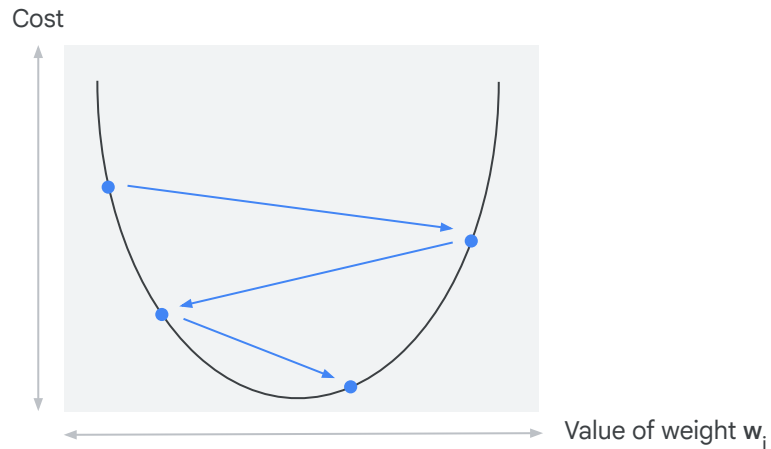
Calculate the derivative to decide the direction



The answer involves the derivative. Let's say you start from the top left. You calculate the derivative of the cost function and find it's negative. This means the angle of the slope is negative and you are at the left side of the curve. To get to the bottom, you must go down and right.

Then, at one point, you are on the right side of the curve, and you calculate the derivative again. This time the value is positive, and you must slide again to the left.

Calculate the derivative to decide the direction

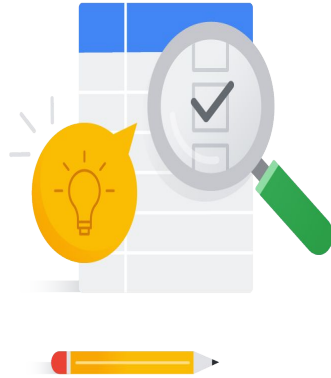


You calculate the derivative of the cost function every time to decide which direction to take. Repeat this process, according to gradient descent, and you will eventually reach the regional bottom.

Finding the bottom

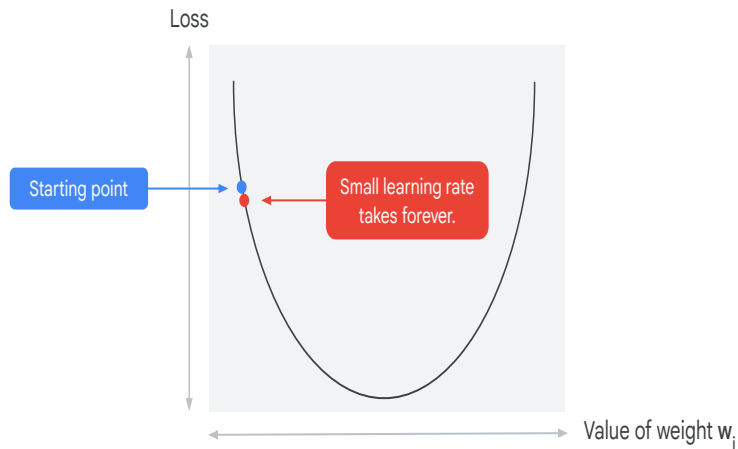
01 Which direction should you take?

02 What size should the steps be?



The second question in finding the bottom is what size should the steps be?

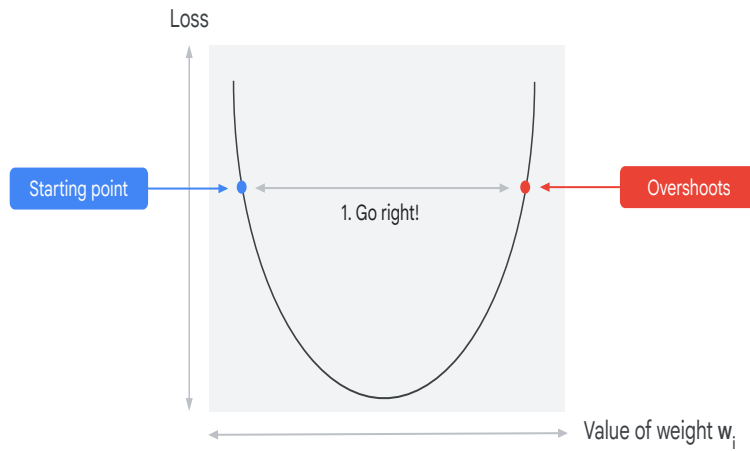
Small step sizes can take a long time to converge



The step size depends on the learning rate, which determines the learning speed of how fast you bounce around to reach the bottom. Step size or “learning rate” is a hyperparameter that is set before training.

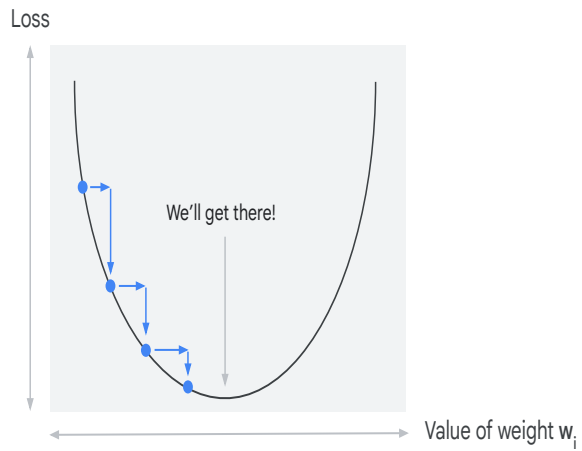
If the step size is too small, your training might take too long.

Large step sizes might never converge

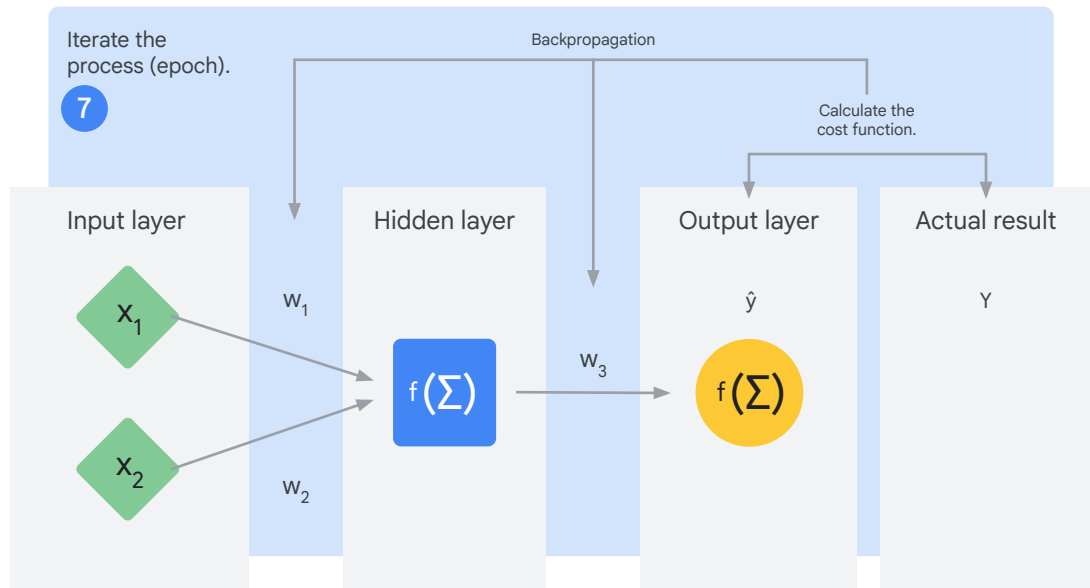


If the step size is too large, you might bounce from wall to wall or even bounce out of the curve entirely, without converging.

Step size is set before training



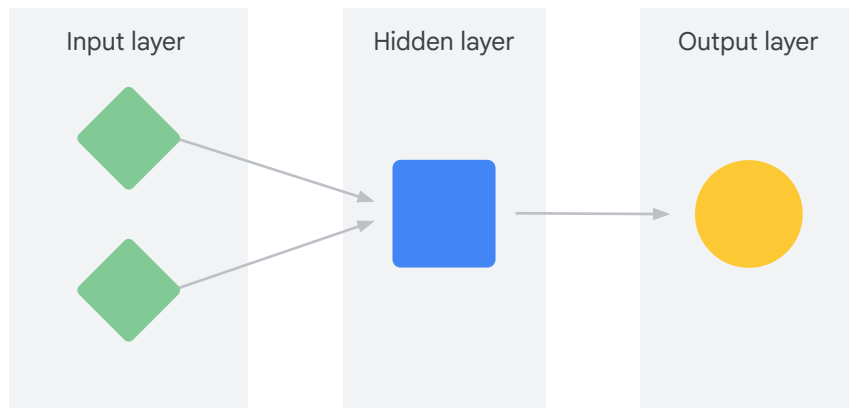
When step size is just right, you're set.



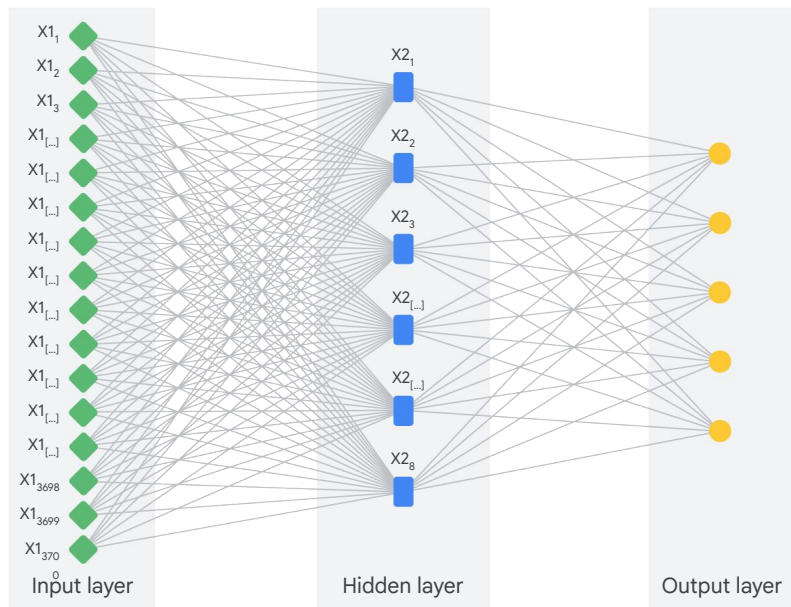
The seventh, and last step, is iteration. One complete pass of the training process from step 1 to step 6 is called an epoch. You can set the number of epochs as a hyperparameter in training.

Weights or parameters are adjusted until the cost function reaches its optimum. You can tell that the cost function has reached its optimum when the value stops decreasing, even after many iterations.

This is how a neural network learns. It iterates the learning by continuously adjusting weights to improve behavior until it reaches the best result. This is similar to a human learning lessons from the past.



We have illustrated a simple example with two input neurons (nodes), one hidden neuron, and one output neuron.



In practice, you might have many neurons in each layer.

Regardless of the number of neurons in the input, hidden, and output layer, the fundamental process of how a neural network learns remains the same.

Concepts you learned about

- ✓ Weights and biases
- ✓ Neurons and layers
- ✓ Activation functions
- ✓ Learning rate
- ✓ Epochs
- ✓ Cost functions or loss functions
- ✓ Backpropagation
- ✓ Gradient descent



Learning about neural networks can be exciting, but also overwhelming with the large number of new terms. Let's take a moment to review them.

Concepts you learned about

- ✓ Weights and biases
- ✓ Neurons and layers
- ✓ Activation functions
- ✓ Learning rate
- ✓ Epochs
- ✓ Cost functions or loss functions
- ✓ Backpropagation
- ✓ Gradient descent



In a neural network, weights and biases are parameters learned by the machine during training. You have no control of the parameters except to set the initial values.

Concepts you learned about

- ✓ Weights and biases
- ✓ Layers and neurons
- ✓ Activation functions
- ✓ Learning rate
- ✓ Epochs
- ✓ Cost functions or loss functions
- ✓ Backpropagation
- ✓ Gradient descent



The number of layers and neurons, activation functions, learning rate, and epochs are hyperparameters, which are decided by a human before training. The hyperparameters determine how a machine learns. For example, the learning rate decides how fast a machine learns and the number of epochs defines how many times the learning iterates.

Normally, data scientists choose the hyperparameters and experiment with them to find the optimum combination. However, if you use a tool like AutoML, it automatically selects the hyperparameters for you and saves you plenty of experiment time.

Concepts you learned about

- ✓ Weights and biases
- ✓ Neurons and layers
- ✓ Activation functions
- ✓ Learning rate
- ✓ Epochs
- ✓ Cost functions or loss functions
- ✓ Backpropagation
- ✓ Gradient descent



You also learned about cost or loss functions, which are used to measure the difference between the predicted and actual value. They are used to minimize error and improve performance.

You use backpropagation to modify the weights and bias if the difference is significant, and gradient descent to decide how to tune the weights and bias and when to stop.

Concepts you learned about

- ✓ Weights and biases
- ✓ Neurons and layers
- ✓ Activation functions
- ✓ Learning rate
- ✓ Epochs
- ✓ Cost functions or loss functions
- ✓ Backpropagation
- ✓ Gradient descent



These terms are your best friends when building an ML model. You'll revisit them in upcoming lessons and labs.

01 AI foundations



02 Generative AI



03 AI development options

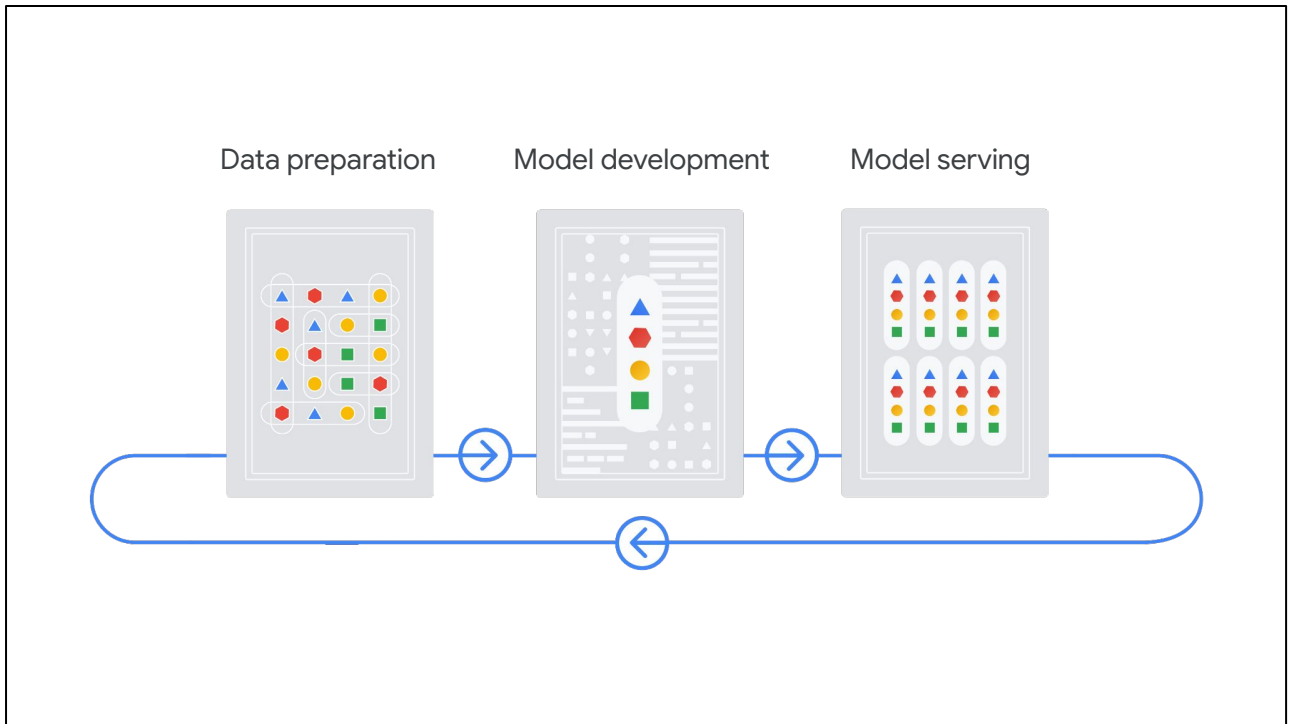


04 AI development workflow



This concludes the module on AI development workflow.

Let's do a quick recap.



In this module, you learned about the three main stages of the machine learning workflow with the help of the restaurant analogy.



1

Data preparation
Upload data.
Engineer features.



2

Model development
Train model.
Evaluate model.



3

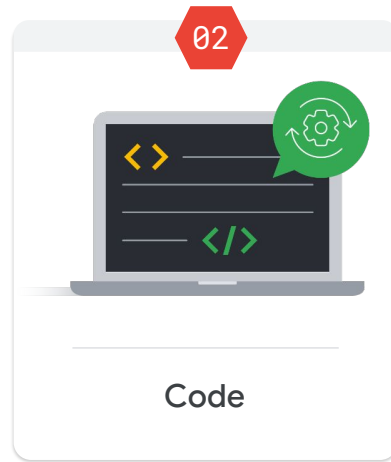
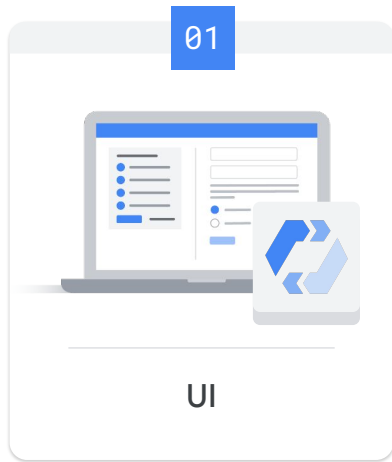
Model serving
Deploy model.
Monitor model.



In stage one, data preparation, you uploaded data and applied feature engineering. This translated to gathering our ingredients and then chopping and prepping them in the kitchen.

In stage two, model development, the model was trained and evaluated. This is where you experimented with the recipes and tasted the meal to ensure that it turned out as expected.

And in the final stage, model serving, the model was deployed and monitored. This translates to serving the meal to customers and adjusting the menu as more people tried and reviewed the dish.



Two ways to build a machine learning model from end to end:

- One is through a **user interface**, like you practiced in the AutoML lab.
- The other is with **code**, which you were shown using prebuilt SDKs with Vertex AI Pipelines. The latter helps you automate the ML pipeline to achieve continuous integration, training, and delivery.

Thanks for joining us in the kitchen, where you explored the full recipe from prepping data to serving models. We can't wait to see you build your own ML models with Google Cloud!

This concludes your journey learning to build AI with Google. We'll see you soon for a short recap of the entire course.